Week6Lecture2026

February 16, 2026

1 Week 6 - Data Visualization

In addition to numbers, you will also want to create visual graphs to help you analyze the data. The three Python libraries you will use for data visualization are pandas, matplotlib, and seaborn. We start with reading from a CSV file and displaying it.

```
import pandas as pd
heart_df = pd.read_csv("heart.csv")
heart_df.head()
```

The **heart.csv** data dictionary defines the meaning of each column. This is very important when loading data for a machine learning model. Knowing as much as possible about the data set will give you an advantage in creating machine learning models. We will cover more next week when we discuss Feature Engineering. | variable | MEANING | |———-|———-| | age | age of patient | | sex | gender | | cp | chest pain type | | trestbps | resting blood pressure | | chol | serum cholestroral in mg/dl | | fbs | fasting blood sugar > 120 mg/dl | | restecg | resting electrocardiographic results (values 0, 1, 2) | | thalach | maximum heart rate achieved | | exang | exercise induced angina | | oldpeak | ST depression induced by exercise relative to rest | | slope | the slope of the peak exercise ST segment | | ca | number of major vessels (0 - 3) colored by flourosopy | | thal | 0 = normal, 1 = fixed defect; 2 = reversable defect The names and social security numbers of the patients wer recently removed from the database, replaced with dummy values | | target | The target column refers to the presence of heart disease in patient. It is integer valued 0 = no disease and 1 = disease. |

[1]:
```
import pandas as pd
heart_df = pd.read_csv("heart.csv")
heart_df.head()
```

[1]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0

```
2   0     3       0
3   1     3       0
4   3     2       0
```

[2]:
```
heart_df["cp"].value_counts()
```

[2]:
```
cp
0    497
2    284
1    167
3     77
Name: count, dtype: int64
```

[3]:
```
heart_df["oldpeak"].value_counts()
```

[3]:
```
oldpeak
0.0    329
1.2     58
1.0     51
0.6     47
0.8     44
1.4     44
1.6     37
0.2     37
1.8     36
2.0     32
0.4     30
0.1     23
2.8     22
2.6     21
3.0     17
1.9     16
1.5     16
3.6     15
0.5     15
2.2     14
4.0     12
2.4     11
0.3     10
3.4     10
0.9     10
3.2      8
2.5      7
2.3      7
1.1      6
4.2      6
4.4      4
```

```
3.1      4
3.8      4
5.6      4
2.9      3
0.7      3
6.2      3
2.1      3
1.3      3
3.5      3
Name: count, dtype: int64
```

[4]:
```
heart_df.index.values
```

[4]:
```
array([   0,    1,    2, …, 1022, 1023, 1024], shape=(1025,))
```

2 Matplotlib Charts

We will create the following charts using the Matplotlib library. We will see how to create these charts with the Pandas libraries. At the end, we will also have a few charts not listed in the Seaborn library.

- Line Plot
- Scatter Plot
- Histogram
- Bar chart/
- Horizontal Bar Chart
- BoxPlot

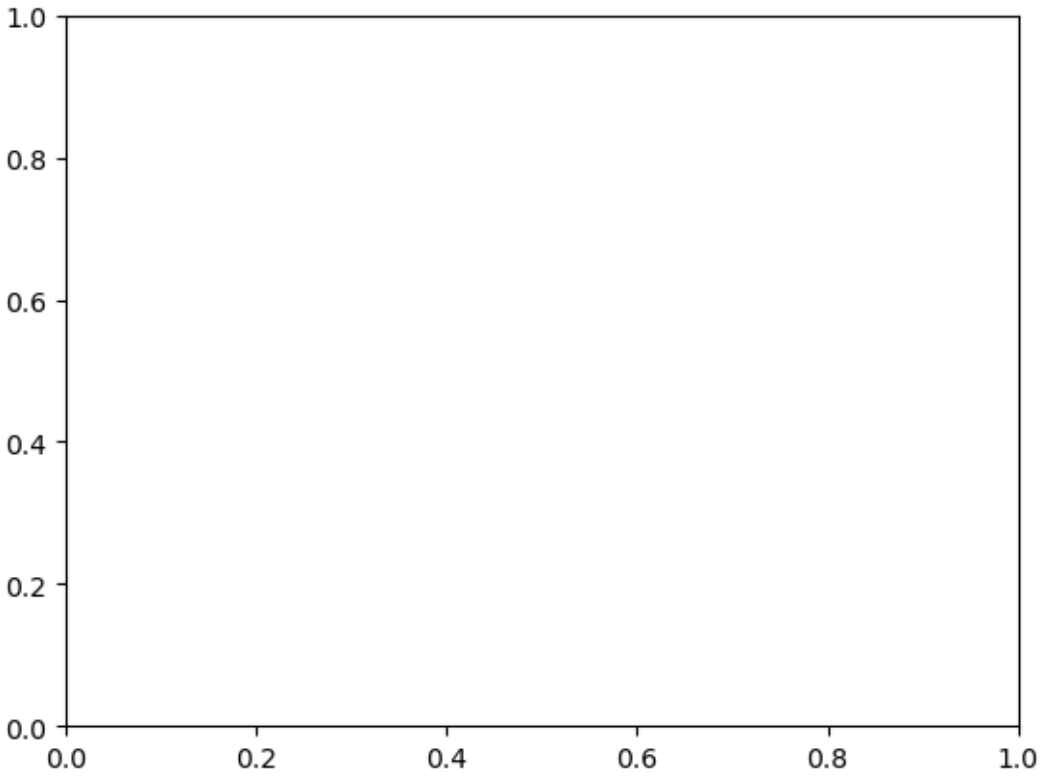
2.0.1 Figures and Axes

First, we must import the matplotlib.pyplot.

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
```

the fig variable show the figure below

[5]:
```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
```

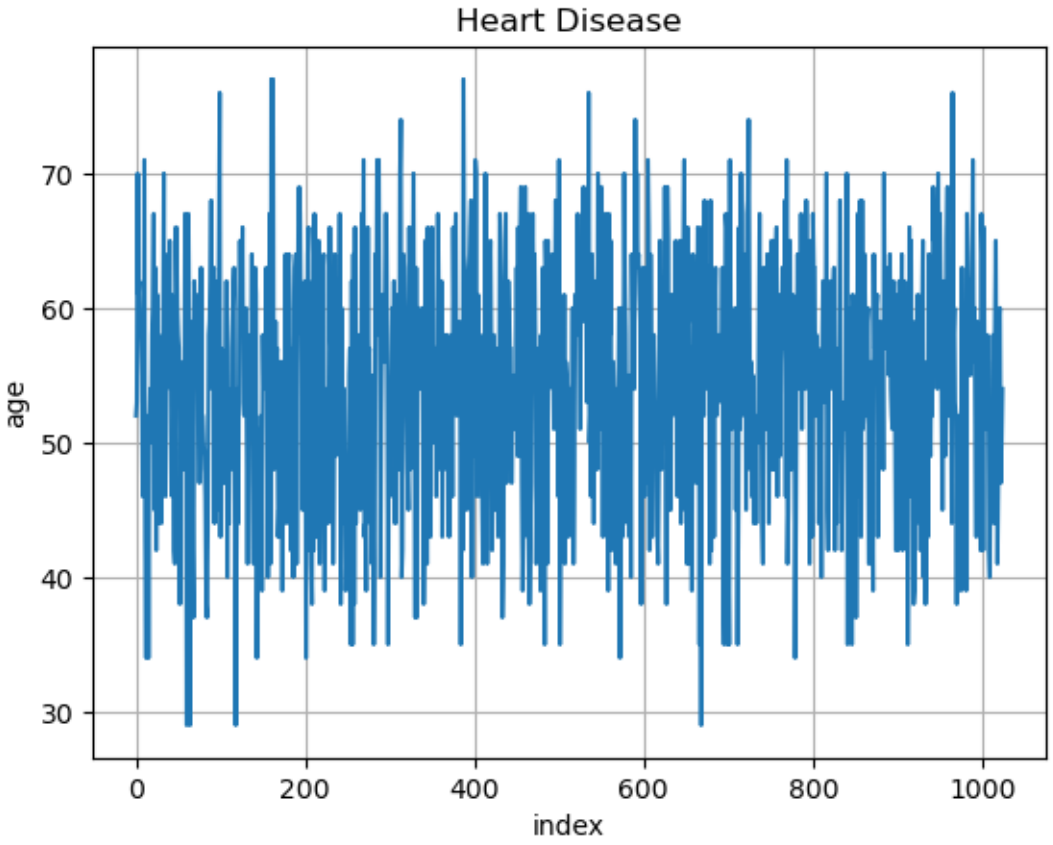


3 Line Plot

A line plot is one of the visualization you can use to show a column in the dataframe to plot the axes.

```
import matplotlib.pyplot as plt
import numpy as np

# Data for plotting
x = heart_df.index.values
y = heart_df["age"]

fig, ax = plt.subplots()
ax.plot(x, y)

ax.set(xlabel = "index", ylabel = "age", title = "Heart Disease")
ax.grid()

# fig.savefig("test.png")  # uncomment to save the graph
plt.show()
```

[6]:
```
import matplotlib.pyplot as plt
#import numpy as np

# Data for plotting
x = heart_df.index.values
y = heart_df["age"]

fig, ax = plt.subplots()
ax.plot(x, y)

ax.set(xlabel = 'index', ylabel = 'age', title = 'Heart Disease')
ax.grid()

fig.savefig("matplotlinechart.png")
plt.show()
```

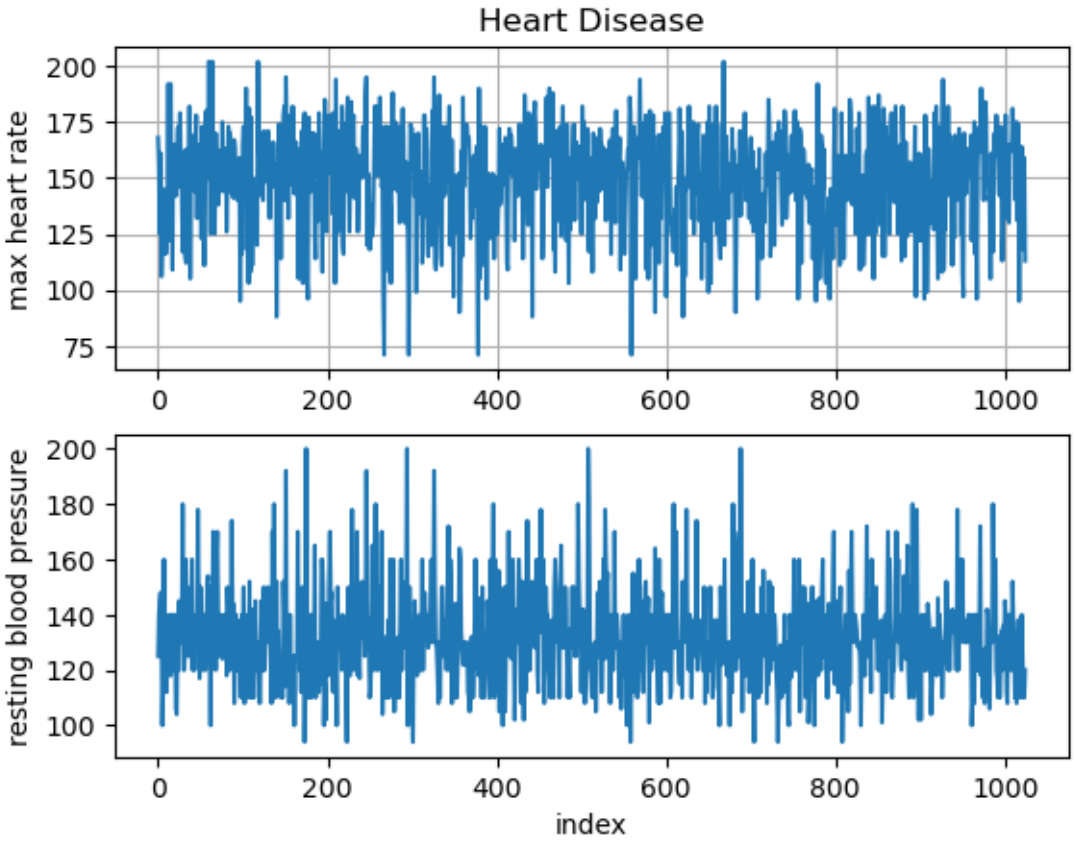


4 Creating mutliple plots

You can create separate line plots with two variables in the dataframe one on top of the other.

```
fig, axes = plt.subplots(2)  # 2  since we have 2 charts
axes[0].plot(heart_df["thalach"])
axes[1].plot(heart_df["trestbps"]);

axes[0].set(xlabel = "", ylabel = "thalach", title = "Heart Disease")
axes[1].set(xlabel = "index", ylabel = "trestbps", title = "")
axes[0].grid()
```

[7]:
```
fig, axes = plt.subplots(2)
axes[0].plot(heart_df["thalach"])
axes[1].plot(heart_df["trestbps"]);

axes[0].set(xlabel = "", ylabel = "max heart rate", title = "Heart Disease")
axes[1].set(xlabel = "index", ylabel = "resting blood pressure", title = "")
axes[0].grid()
```



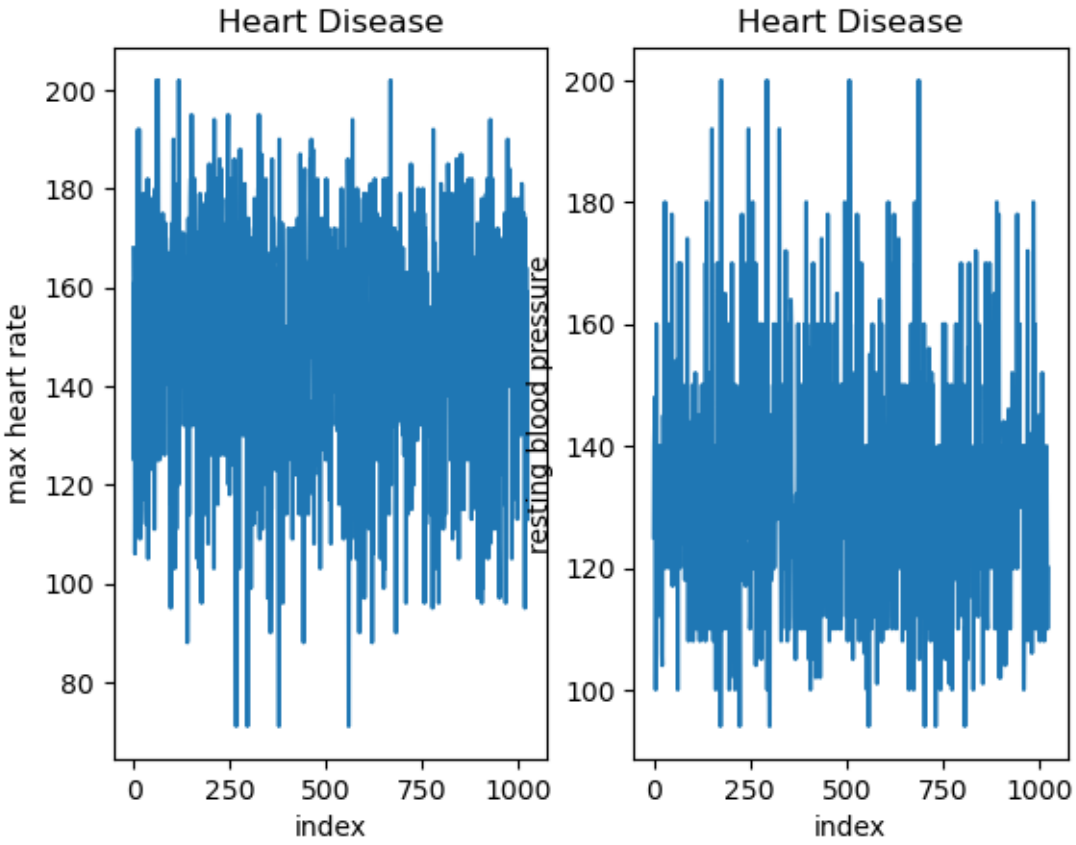
5 plots side by side

```
fig, axes = plt.subplots(1, 2)
```

```
axes[0].plot(heart_df["thalach"])
axes[1].plot(heart_df["trestbps"])

axes[0].set(xlabel = "index", ylabel = "thalach", title = "Heart Disease")
axes[1].set(xlabel = "index", ylabel = "trestbps", title = "Heart Disease")
```

[8]:
```
fig, axes = plt.subplots(1, 2)

axes[0].plot(heart_df["thalach"])
axes[1].plot(heart_df["trestbps"])

axes[0].set(xlabel = "index", ylabel = "max heart rate", title = "Heart␣
 ↪Disease")
axes[1].set(xlabel = "index", ylabel = "resting blood pressure", title = "Heart␣
 ↪Disease")
```

[8]:
```
[Text(0.5, 0, 'index'),
 Text(0, 0.5, 'resting blood pressure'),
 Text(0.5, 1.0, 'Heart Disease')]
```

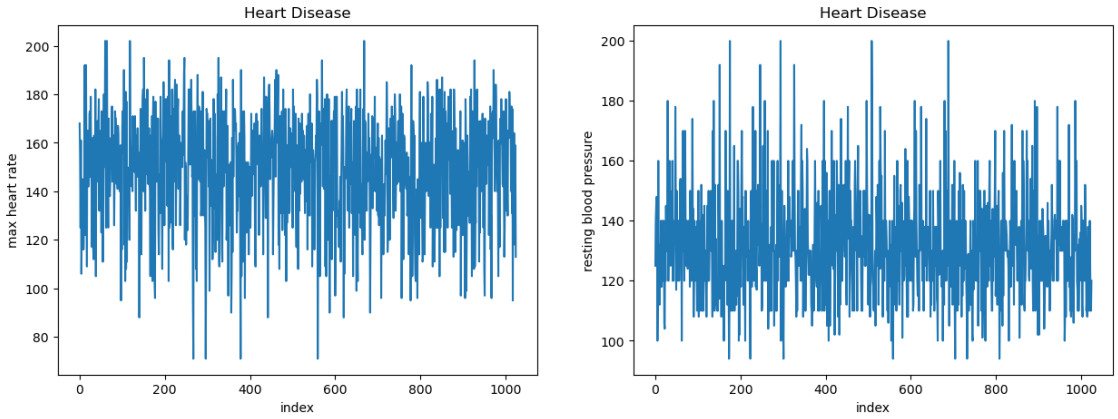


6 Set the figsize

```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].plot(heart_df["thalach"])
axes[1].plot(heart_df["trestbps"])
```

[9]:
```
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

axes[0].plot(heart_df["thalach"])
axes[1].plot(heart_df["trestbps"])

axes[0].set(xlabel = "index", ylabel = "max heart rate", title = "Heart␣
↪Disease")
axes[1].set(xlabel = "index", ylabel = "resting blood pressure", title = "Heart␣
↪Disease")
```

[9]:
```
[Text(0.5, 0, 'index'),
 Text(0, 0.5, 'resting blood pressure'),
 Text(0.5, 1.0, 'Heart Disease')]
```



7 Histogram

Another plot that is used in data analysis is a histogram. To create a histogram with 10 bins follow the coding example below.

```
fig, ax = plt.subplots()
ax.hist(heart_df["thalach"], color = "g", edgecolor ='black')
ax.set(xlabel = "thalach", title = "Histogram");
```

8 The basic built-in colors.

8.1 - b : blue

8.2 - g : green

8.3 - r : red

8.4 - c : cyan

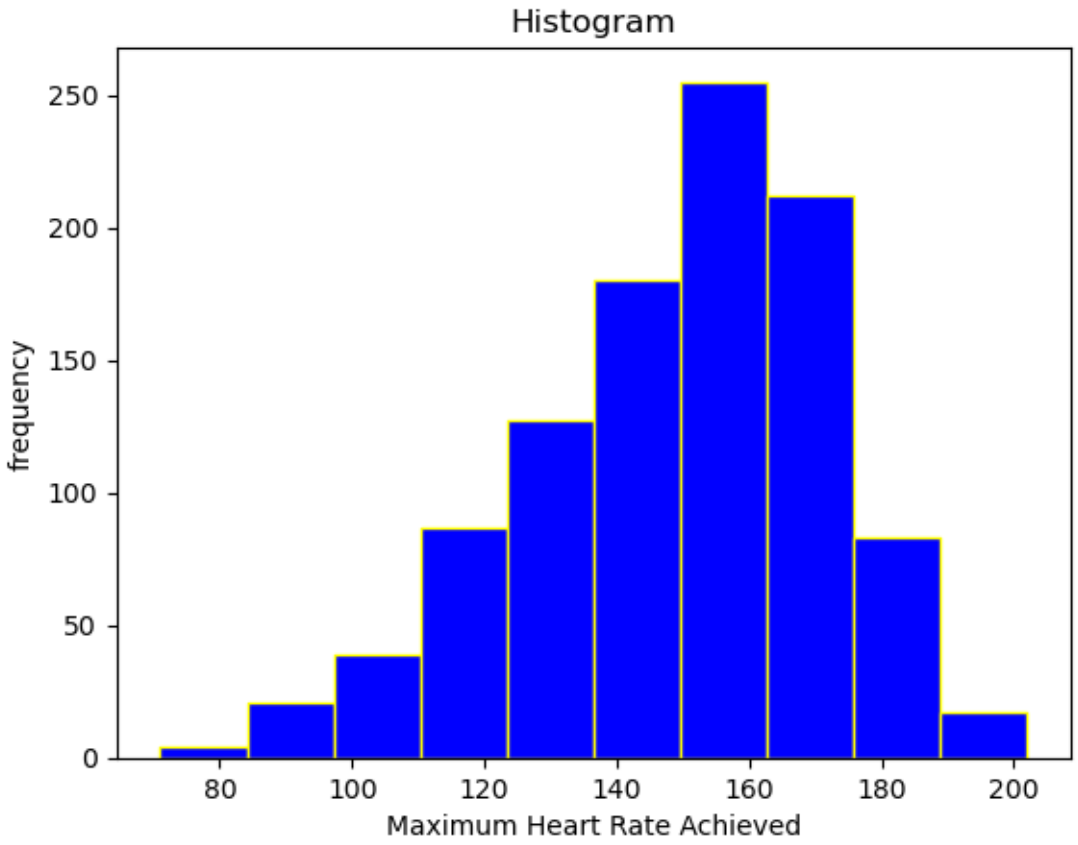
8.5 - m : magenta

8.6 - y : yellow

8.7 - k : black

8.8 - w : white

[10]:
```
fig, ax = plt.subplots()
ax.hist(heart_df["thalach"], color = "b", edgecolor ='yellow')
ax.set(xlabel = "Maximum Heart Rate Achieved", ylabel = "frequency", title =␣
↪"Histogram");
```

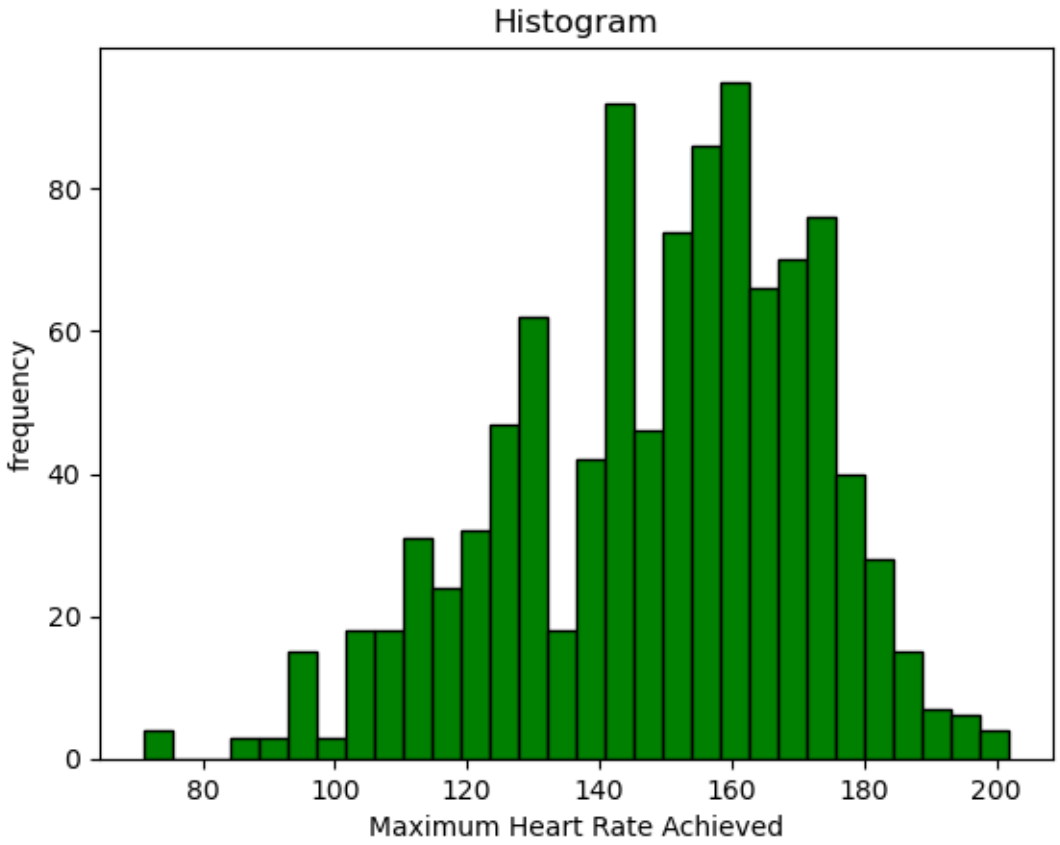


9 Change bin size

You may want to increase the number bins in your histogram. To do so use the code below.

```
fig, ax = plt.subplots()
ax.hist(heart_df["thalach"], color = "b", edgecolor="black", bins= 30)
    ax.set(xlabel = "thalach", title = "Histogram");
```

[11]:
```
fig, ax = plt.subplots()
ax.hist(heart_df["thalach"], color = "g", edgecolor="black", bins= 30)
ax.set(xlabel = "Maximum Heart Rate Achieved", ylabel = "frequency", title =␣
 ↪"Histogram");
```

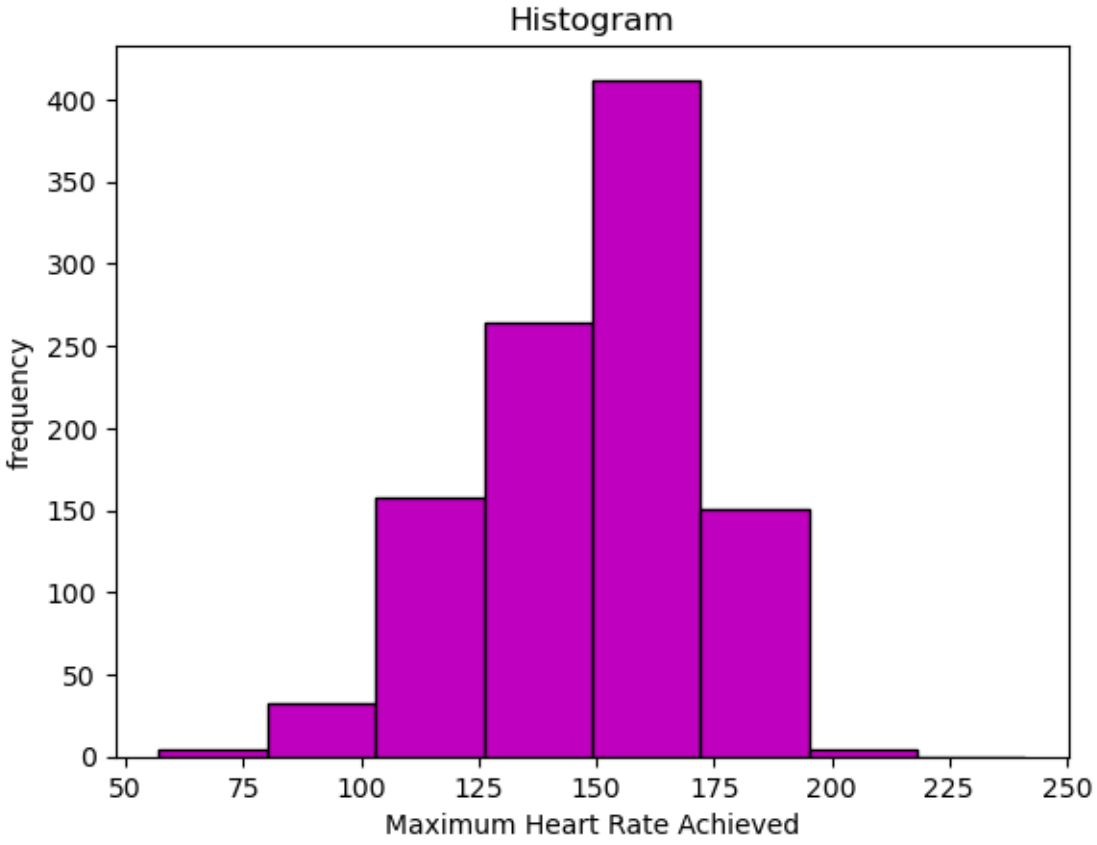


10 Changing bins using the Empirical Rule

10.0.1 The empirical rule states that if the distribution is normal.

- 68% is within mean - 1 * standard deviation and mean + 1 * standard deviation
- 95% is within mean - 2 * standard deviation and mean + 2 * standard deviation
- 99.7% is within mean - 3 * standard deviation and mean + 3 * standard deviation

[12]:
```
mu = round(heart_df["thalach"].mean().item(), 2)
std_1 = round(heart_df["thalach"].std().item(), 2)
L1 = [mu - 4 * std_1, mu - 3 * std_1, mu - 2 * std_1, mu - 1 * std_1, mu, mu +␣
 ↪1 * std_1, mu + 2 * std_1, mu + 3 * std_1, mu + 4 * std_1]
fig, ax = plt.subplots()
ax.hist(heart_df["thalach"], color = "m", edgecolor="black", bins= L1)
ax.set(xlabel = "Maximum Heart Rate Achieved", ylabel = "frequency", title =␣
 ↪"Histogram");
```



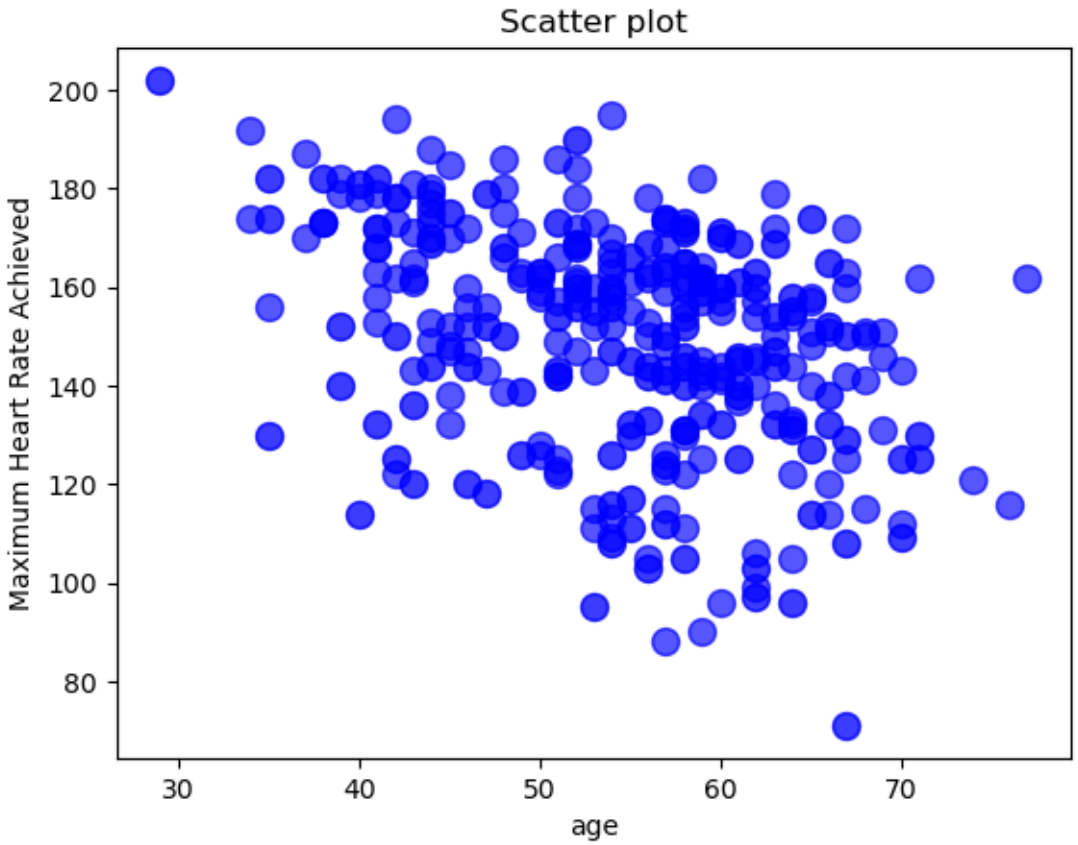
11 Scatterplot

A scatterplot requires two variables. In this case, we will look at "age" and "thalach". Use the code below to create a scatterplot.

```
fig, ax = plt.subplots()
ax.scatterplot(x = heart_df["age"], y = heart_df["thalach"], alpha = 0.3, s = 100, c = "blue")
ax.set(title = "Heart Disease", xlabel = "age", ylabel = "thalach");
```

[13]:
```
fig, ax = plt.subplots()
```

```
ax.scatter(x = heart_df["age"], y = heart_df["thalach"], alpha = 0.3, s = 100,␣
 ↪c = "blue")
ax.set(title = "Scatter plot", xlabel = "age", ylabel = "Maximum Heart Rate␣
 ↪Achieved");
```



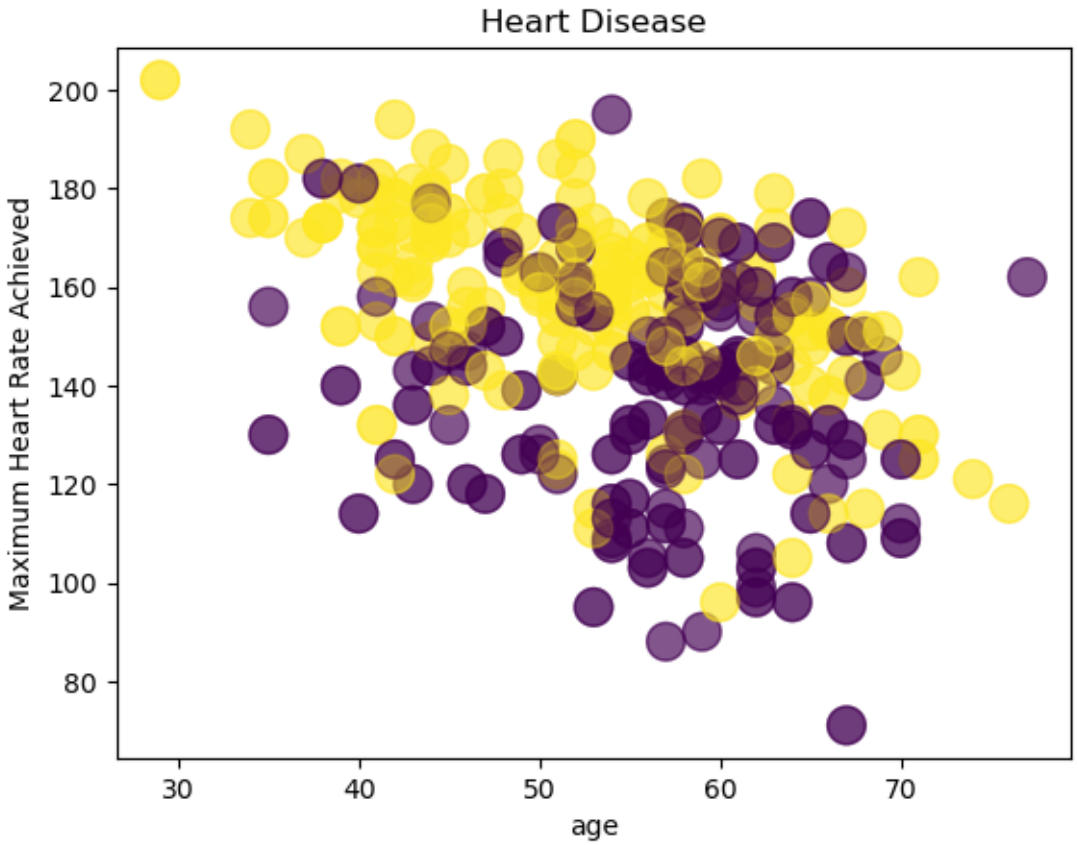
12 Auto-set the colors based on class (target variable)

12.0.1 When you have a classification variable to predict, you want to know whether there are distinct regions for the classification groups. Here, the target column has the category you want to predict. In this case, the target is the presence or absence of Heart Disease.

```
fig, ax = plt.subplots()

ax.scatter(heart_df["age"], heart_df["thalach"], alpha = 0.3, s =200, c = heart_df["target"])
ax.set(title = "Heart Disease", xlabel = "age", ylabel = "Maximum Heart Rate Achieved");
```

[14]:
```
fig, ax = plt.subplots()
```

```
ax.scatter(heart_df["age"], heart_df["thalach"], alpha = .3, s =200, c =␣
 ↪heart_df["target"])
ax.set(title = "Heart Disease", xlabel = "age", ylabel = "Maximum Heart Rate␣
 ↪Achieved");
```



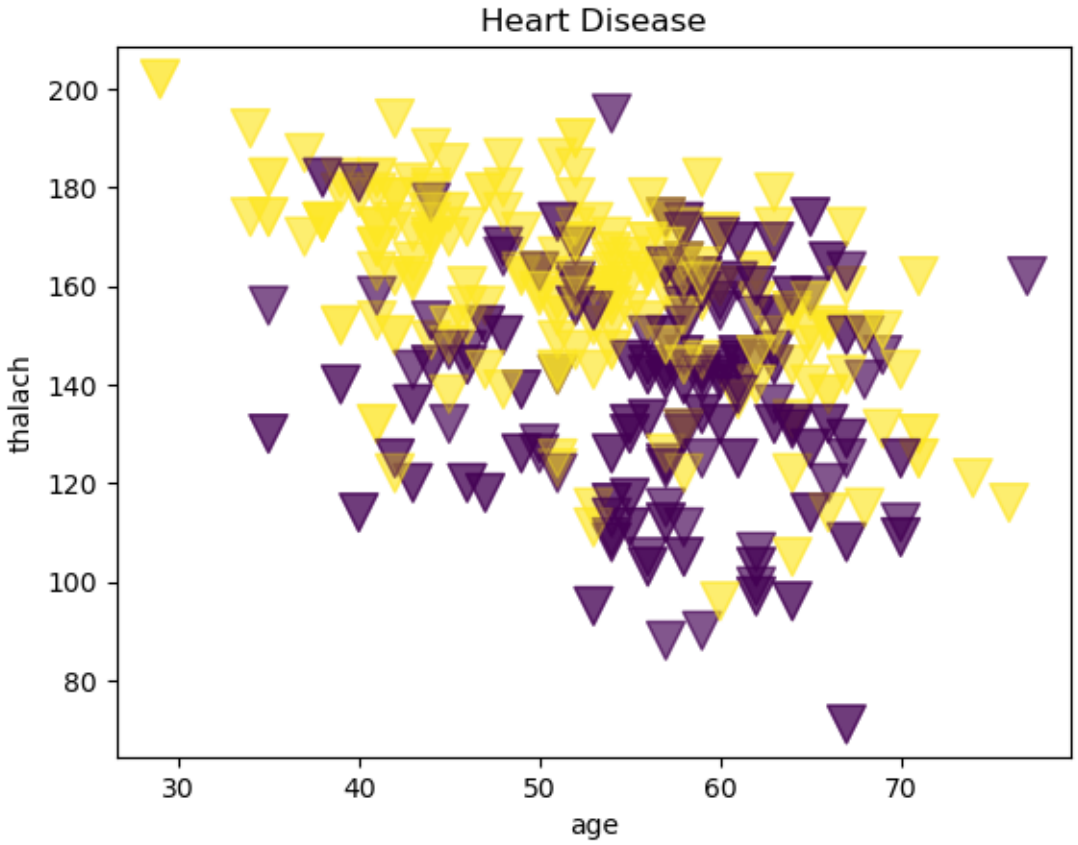
[ ]:

13 set the marker (shape)

```
#sample markers: "o" (default), "v", "^", "x", "P", "d", "."
ax.scatter(heart_df["age"], df["thalach"], alpha = 0.3, s = 200, c = heart_df["target"], marke
ax.set(title = "Heart Disease", xlabel = "age", ylabel = "thalach");
plt.savefig("my_plot.pdf")
```

[15]:
```
#sample markers: "o" (default), "v", "^", "x", "P", "d", "."
fig, ax = plt.subplots()

ax.scatter(heart_df["age"], heart_df["thalach"], alpha = 0.3, s = 200, c =␣
 ↪heart_df["target"], marker = "v")
```

```
ax.set(title = "Heart Disease", xlabel = "age", ylabel = "thalach");
plt.savefig("my_plot.pdf")
```



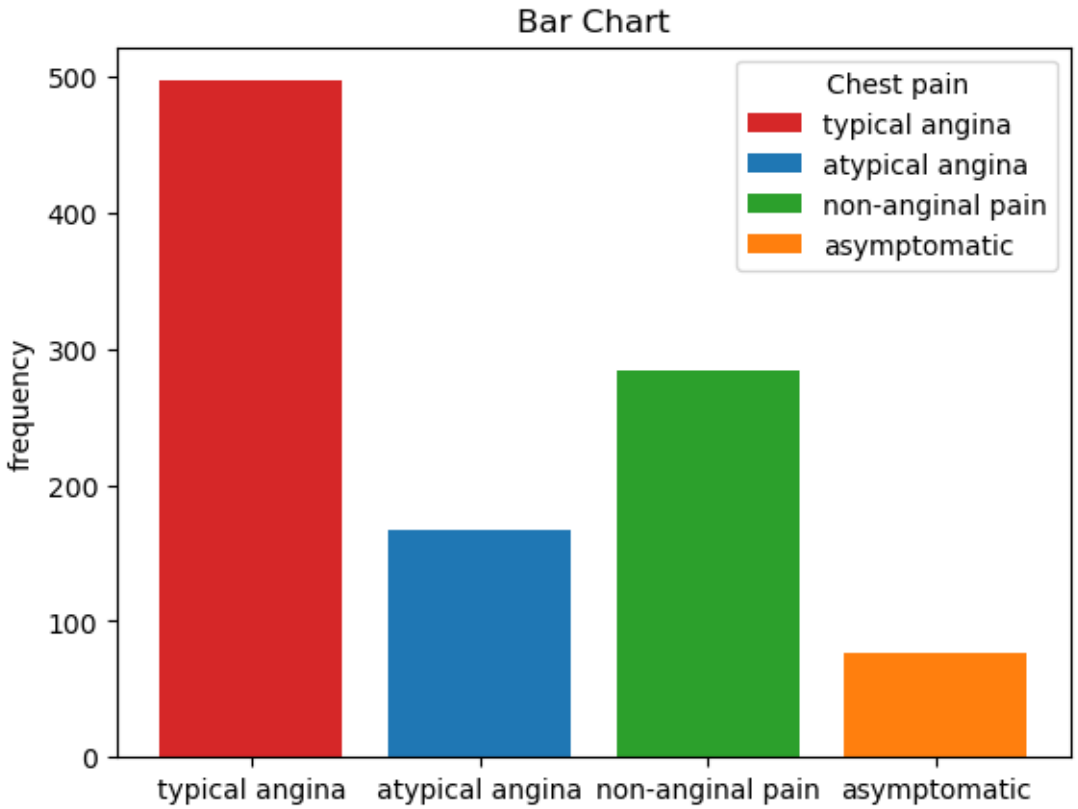
14 Bar Chart

When you have categorical data you may want to create a bar chart to see the frequency of each categorical value. A categorical variable cp, meaning chest pain. Some research found the values in the column means. | Chest pain cp | MEANING | |————|————| | 0 | Typical angina | | 1 | atypical angina | | 2 | non-anginal pain | | 3 | asymptomatic |

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

gh = heart_df["cp"].value_counts().sort_index()
y = gh.values
x = gh.index

bar_colors = ['tab:red', 'tab:blue', 'tab:green', 'tab:orange']
x_labels = ["typical angina", "atypical angina", "non-anginal pain", "asymptomatic"]
```

```
ax.bar(x_labels, y, label = x_labels, color = bar_colors)
ax.set_ylabel("frequency")
ax.set_title("Bar Chart")
ax.legend(title = "Chest pain")
plt.show()
```

[16]:
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

gh = heart_df["cp"].value_counts().sort_index()
y = gh.values
x = gh.index

bar_colors = ['tab:red', 'tab:blue', 'tab:green', 'tab:orange']
x_labels = ["typical angina", "atypical angina", "non-anginal pain",␣
 ↪"asymptomatic"]
ax.bar(x_labels, y, label = x_labels, color = bar_colors)
ax.set_ylabel("frequency")
ax.set_title("Bar Chart")
ax.legend(title = "Chest pain")
plt.show()
```

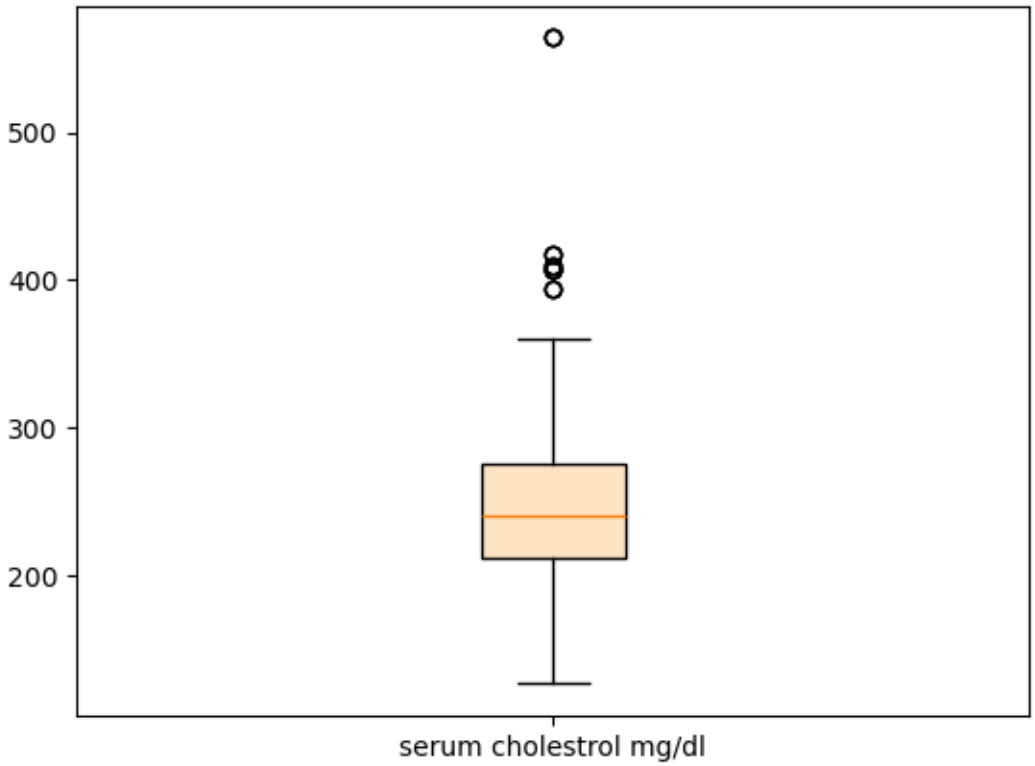


15 Box Plot - Column

You can create a box plot to see if there are any outliers in your data set column. Let's look at the Boxplot for serum cholestroral in mg/dl.

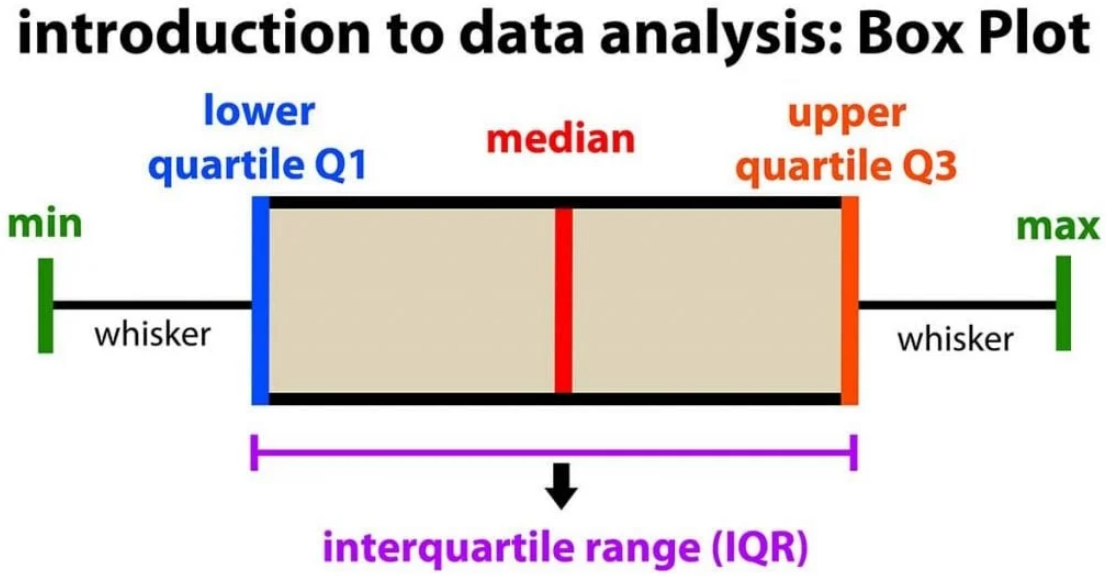
```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import cbook

data = heart_df["chol"]
fig, ax1 = plt.subplots()

# single boxplot call
ax1.boxplot(data, tick_labels=['serum cholestrol mg/dl'],
            patch_artist=True, boxprops={'facecolor': 'bisque'})

# separate calculation of statistics and plotting
stats = cbook.boxplot_stats(data, labels=['serum cholestrol mg/dl'])
```

[17]:
```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import cbook

data = heart_df["chol"]
fig, ax1 = plt.subplots()

# single boxplot call
ax1.boxplot(data, tick_labels=['serum cholestrol mg/dl'],
            patch_artist=True, boxprops={'facecolor': 'bisque'})

# separate calculation of statistics and plotting
stats = cbook.boxplot_stats(data, labels=['serum cholestrol mg/dl'])
```



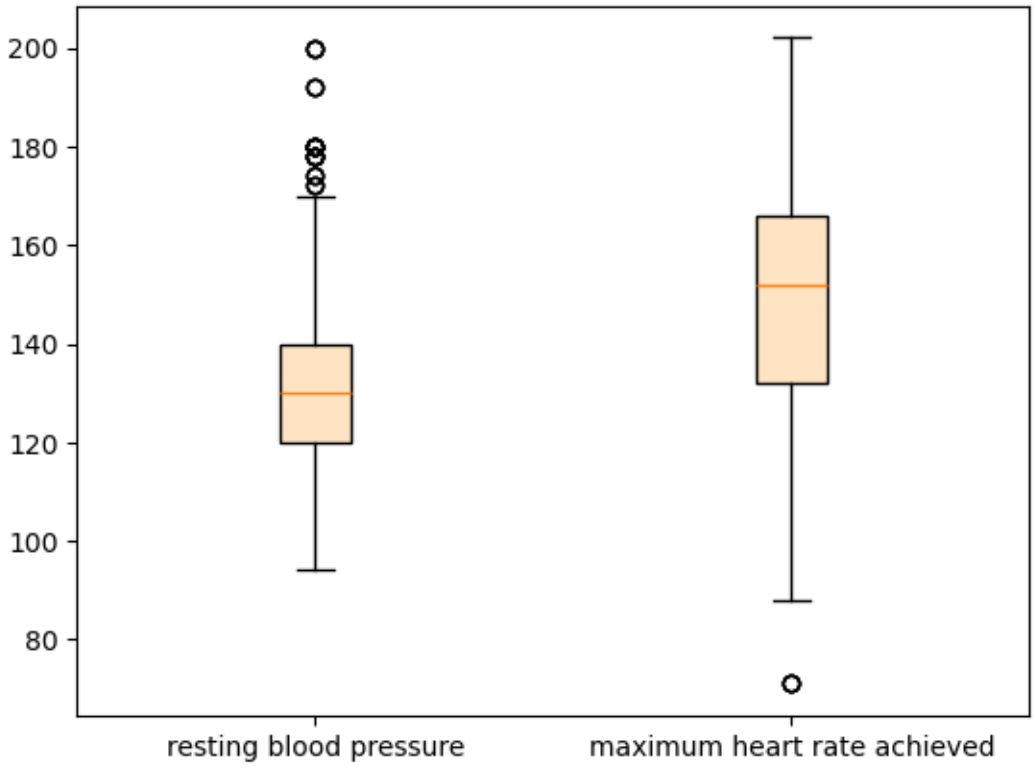
16 2 or more Box Plots

You may want to compare multiple boxplots in same graph. Make sure they have a similar range of values so you can see all the elements of a box plot.



```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import cbook

data = heart_df[["trestbps", "thalach"]]
fig, ax1 = plt.subplots()

# single boxplot call
ax1.boxplot(data, tick_labels=["resting blood pressure", "maximum heart rate achieved"],
            patch_artist=True, boxprops={'facecolor': 'bisque'})

# separate calculation of statistics and plotting
stats = cbook.boxplot_stats(data, labels=["resting blood pressure", "maximum heart rate achieve
```

[18]:
```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib import cbook

data = heart_df[["trestbps", "thalach"]]
fig, ax1 = plt.subplots()

# single boxplot call
ax1.boxplot(data, tick_labels=["resting blood pressure", "maximum heart rate␣
 ↪achieved"],
            patch_artist=True, boxprops={'facecolor': 'bisque'})

# separate calculation of statistics and plotting
stats = cbook.boxplot_stats(data, labels=["resting blood pressure", "maximum␣
 ↪heart rate achieved"])
```



17 Pandas

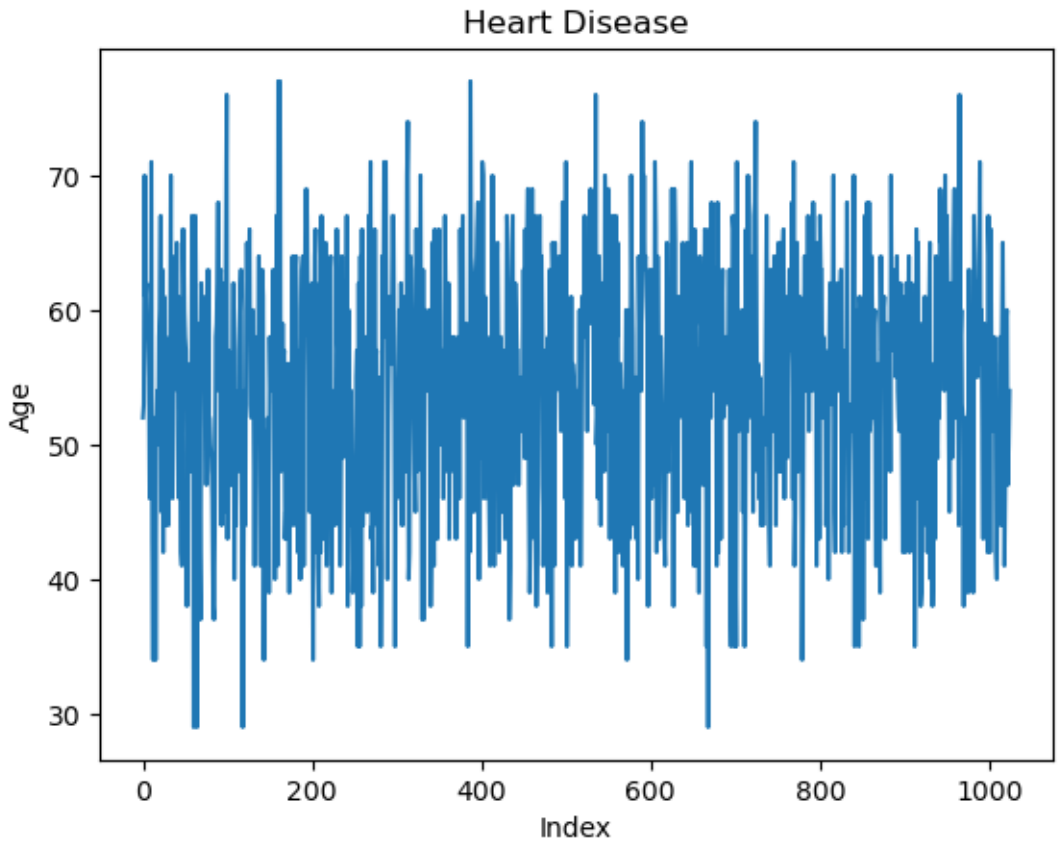
Now I will show you how to create the following graphs in pandas - Line Plot - Histogram - Scatterplot - Bar chart - Box plot

Let's get started with the Line Plot. Here are the commands to create a Line Plot.

```
heart_df["age"].plot.line(xlabel="Index", ylabel="Age", title="Heart Disease")
```

[19]:
```
# Pass labels directly into the plot function
heart_df["age"].plot.line(xlabel="Index", ylabel="Age", title="Heart Disease")
```

[19]:
```
<Axes: title={'center': 'Heart Disease'}, xlabel='Index', ylabel='Age'>
```

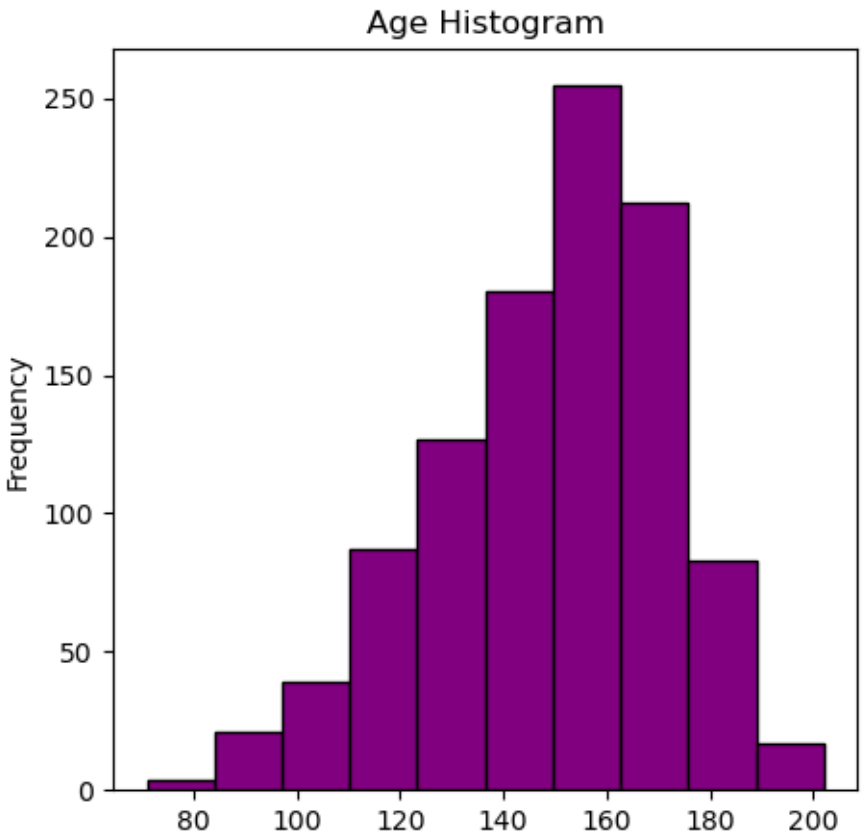


18 Histogram

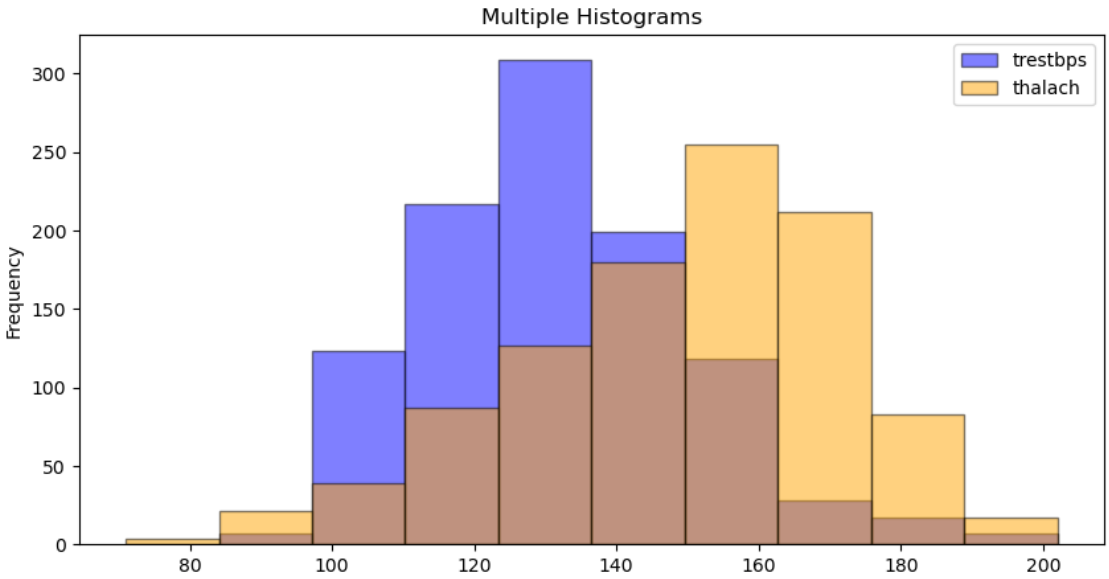
To create a histogram for one column use the commands below.

```
heart_df["thalach"].plot( kind = "hist", color = "purple", edgecolor = "black", title = "Age Hi
heart_df.plot(y = "age", kind = "hist", color = "purple", edgecolor = "black", title = "Age His
```

[20]:
```
heart_df["thalach"].plot( kind = "hist", color = "purple", edgecolor = "black",␣
↪title = "Age Histogram", figsize = (5, 5));
```



[21]:
```
heart_df.plot(y = ["trestbps", "thalach"], kind = "hist", color = ["blue",␣
↪"orange"], edgecolor = "black", title = "Multiple Histograms", figsize =␣
↪(10, 5), alpha = 0.5);
```

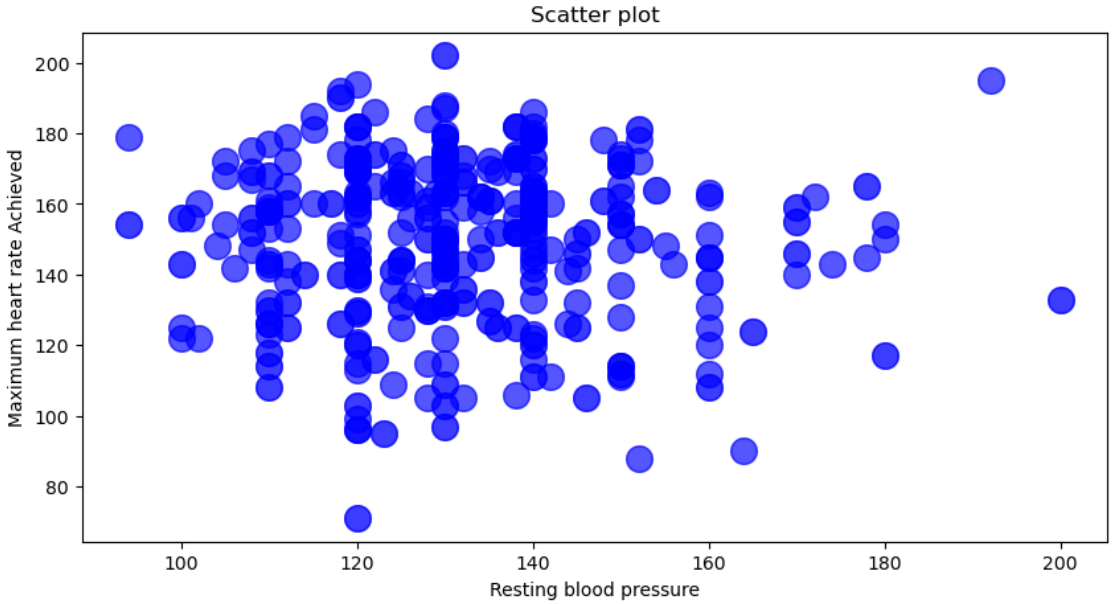


19 Scatterplots

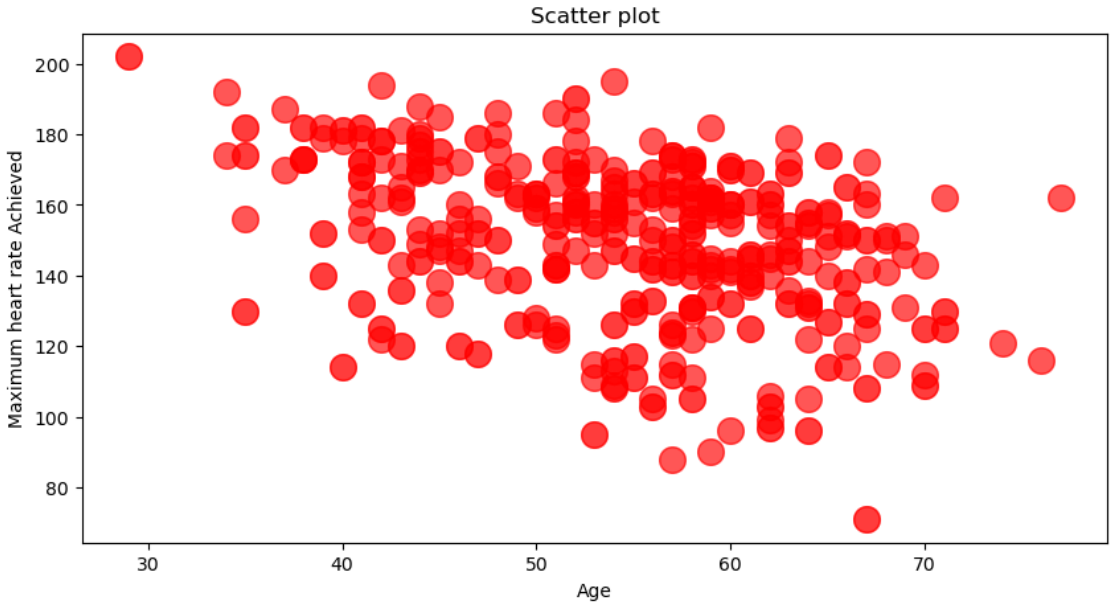
To create a scatterplot using two numeric column such as “trestbps” and “thalach”.

```
heart_df[["trestbps", "thalach"]].plot(x = "trestbps", y = "thalach", kind= "scatter", xlabel =
                                       ylabel = "Maximum heart rate Achieved", title = "Scatte
                                       alpha = 0.3, figsize = (10, 5));
```

[22]:
```
heart_df[["trestbps", "thalach"]].plot(x = "trestbps", y = "thalach", kind=␣
↪"scatter", xlabel = "Resting blood pressure", ylabel = "Maximum heart rate␣
↪Achieved", title = "Scatter plot", s = 200, color = "blue", alpha = 0.3,␣
↪figsize = (10, 5));
```



[23]:
```
heart_df[["age", "thalach"]].plot(x = "age", y = "thalach", kind= "scatter",
                                  xlabel = "Age", ylabel = "Maximum heart␣
↪rate Achieved",
                                  title = "Scatter plot", s = 200, color =␣
↪"red", alpha = 0.3, figsize = (10, 5));
```

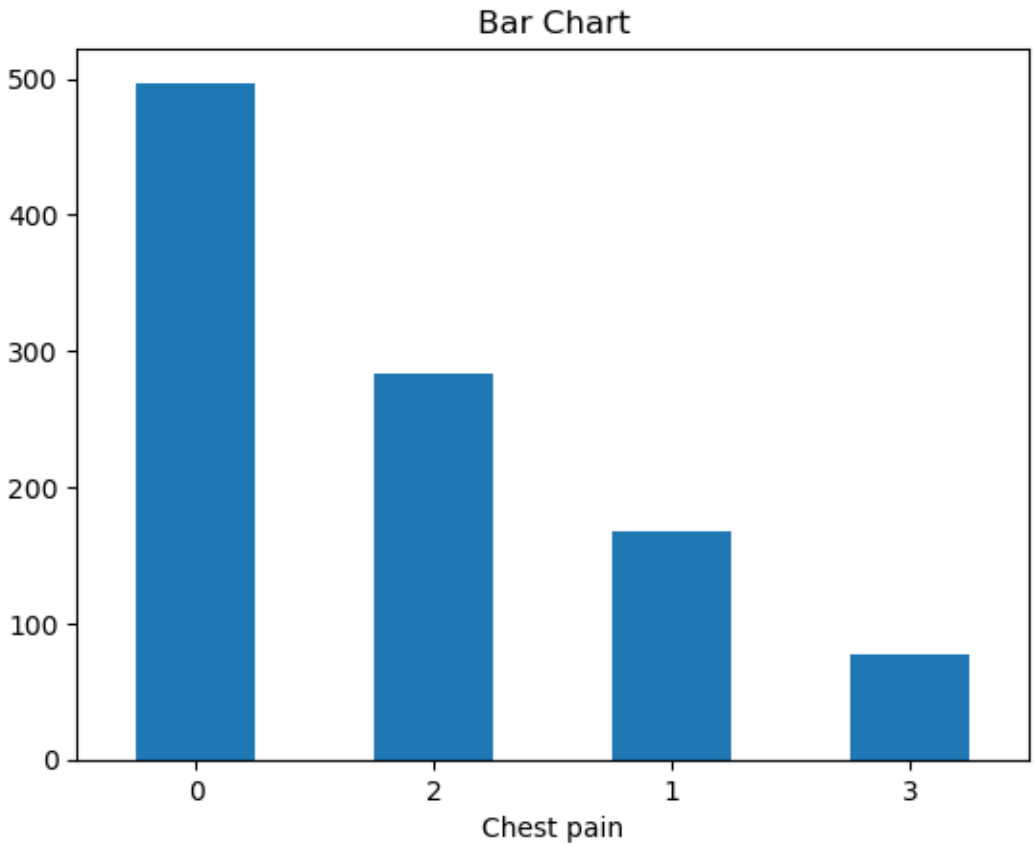


20 Bar Chart

To create a bar chart.

```
heart_df["cp"].value_counts().plot(kind = "bar", xlabel = "Chest pain", title = "Bar Chart", r
```

[24]:
```
heart_df["cp"].value_counts().plot(kind = "bar", xlabel = "Chest pain", title =␣
↪"Bar Chart", rot = "horizontal");
```

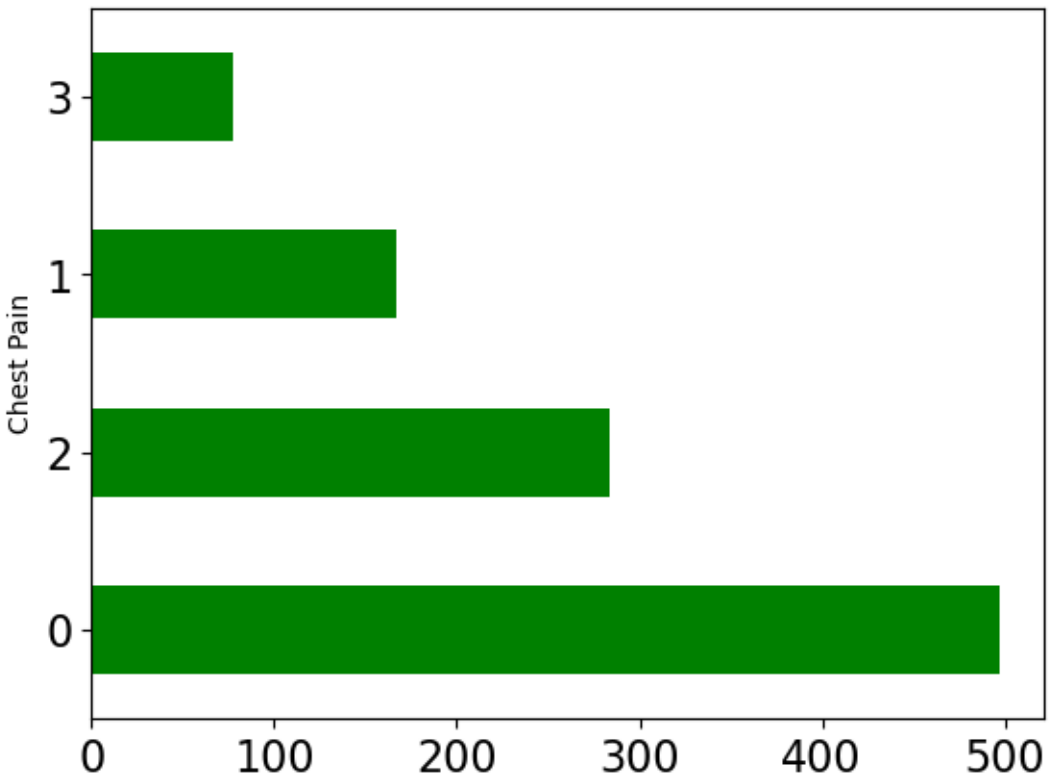


21 Horizontal Bar Chart

Use this command.

```
hf = heart_df["cp"].value_counts()
y = "Chest Pain"
hf.plot(kind = "barh", ylabel = y, color = "green", fontsize = 16);
```

[25]:
```
hd = heart_df["cp"].value_counts()
y = "Chest Pain"
hd.plot(kind = "barh", ylabel = y, color = "green", fontsize = 16);
```

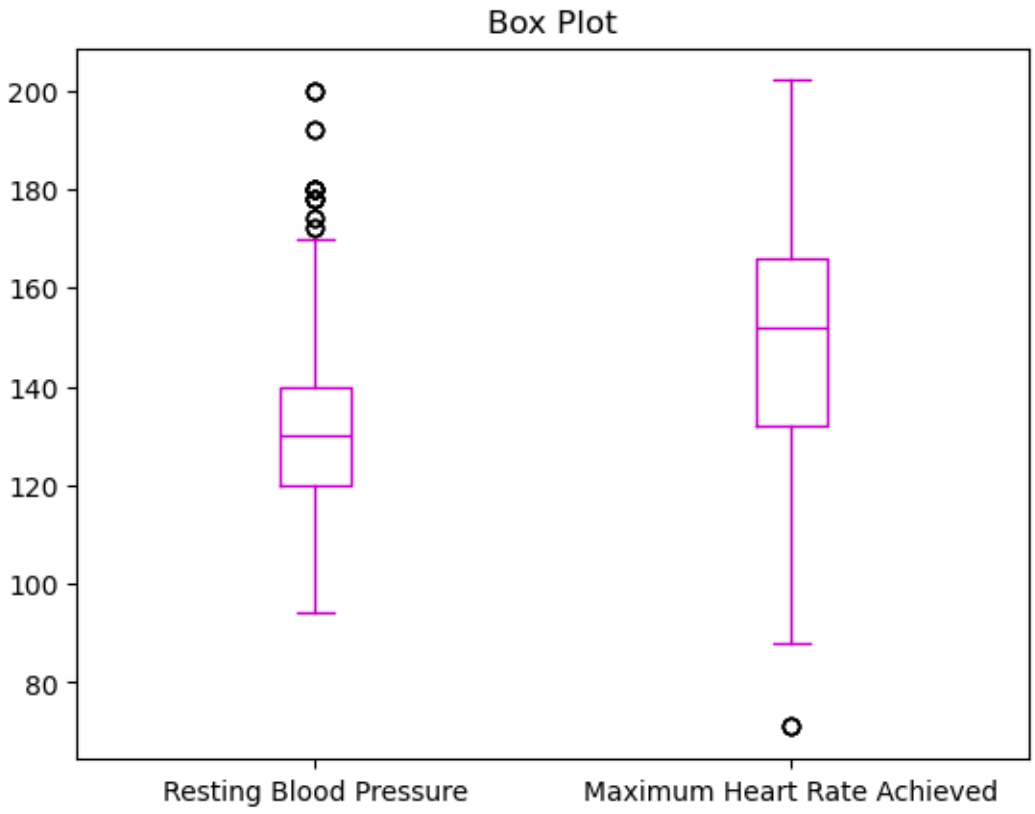


22 Box Plot - Pandas

```
b1 = "Resting Blood Pressure"
b2 = "Maximum Heart Rate Achieved"
blabels = {"trestbps" : b1, "thalach" : b2}
hd = heart_df[["trestbps", "thalach"]].rename(columns = blabels)
columnx = ["Resting Blood Pressure", "Maximum Heart Rate Achieved"]
hd.plot( y = columnx, kind = "box", color = "m", title = "Box plot" )
```

[26]:
```
b1 = "Resting Blood Pressure"
b2 = "Maximum Heart Rate Achieved"
blabels = {"trestbps" : b1, "thalach" : b2}
hd = heart_df[["trestbps", "thalach"]].rename(columns = blabels)
columnx = ["Resting Blood Pressure", "Maximum Heart Rate Achieved"]
hd.plot( y = columnx, kind = "box", color = "m", title = "Box Plot" )
```

[26]:
```
<Axes: title={'center': 'Box Plot'}>
```

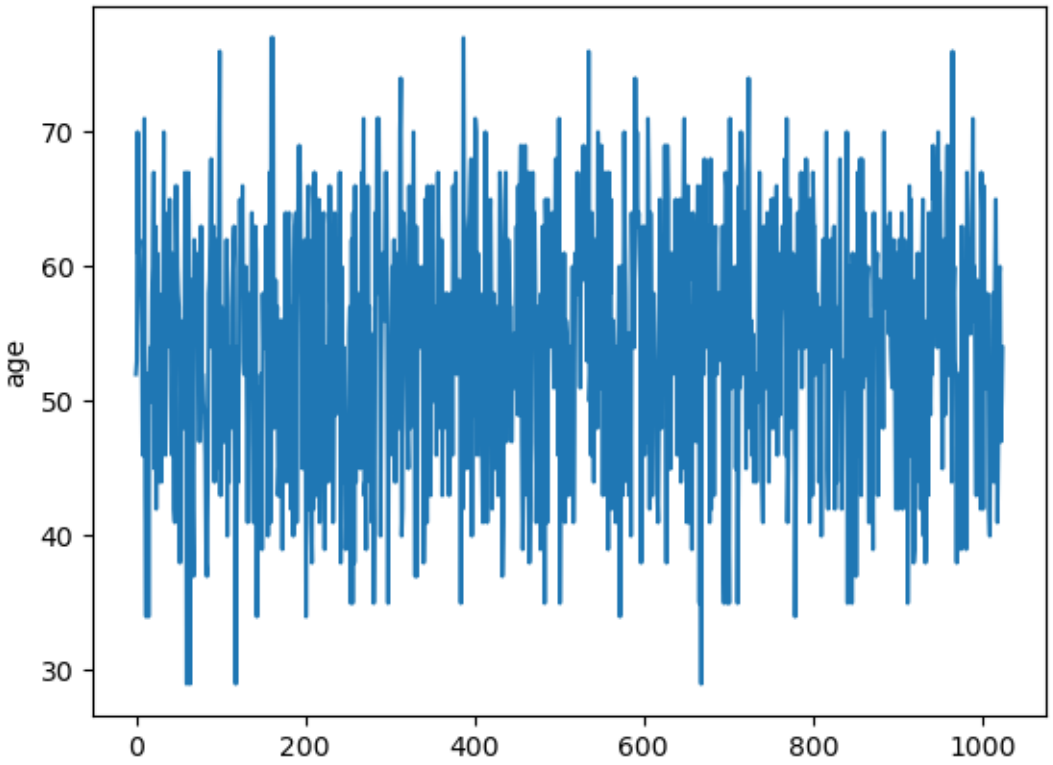


23 Seaborn

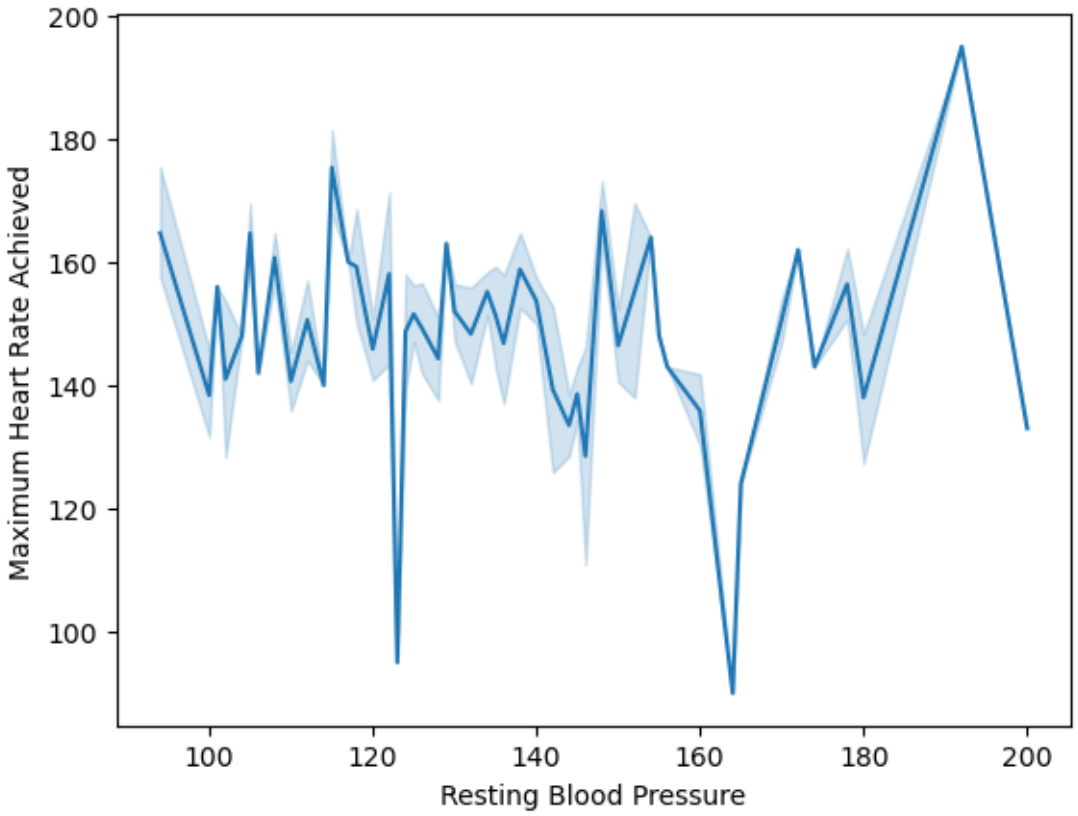
23.1 Line Plot

```
import seaborn as so
so.lineplot(data = heart_df["trestbps"]);
```

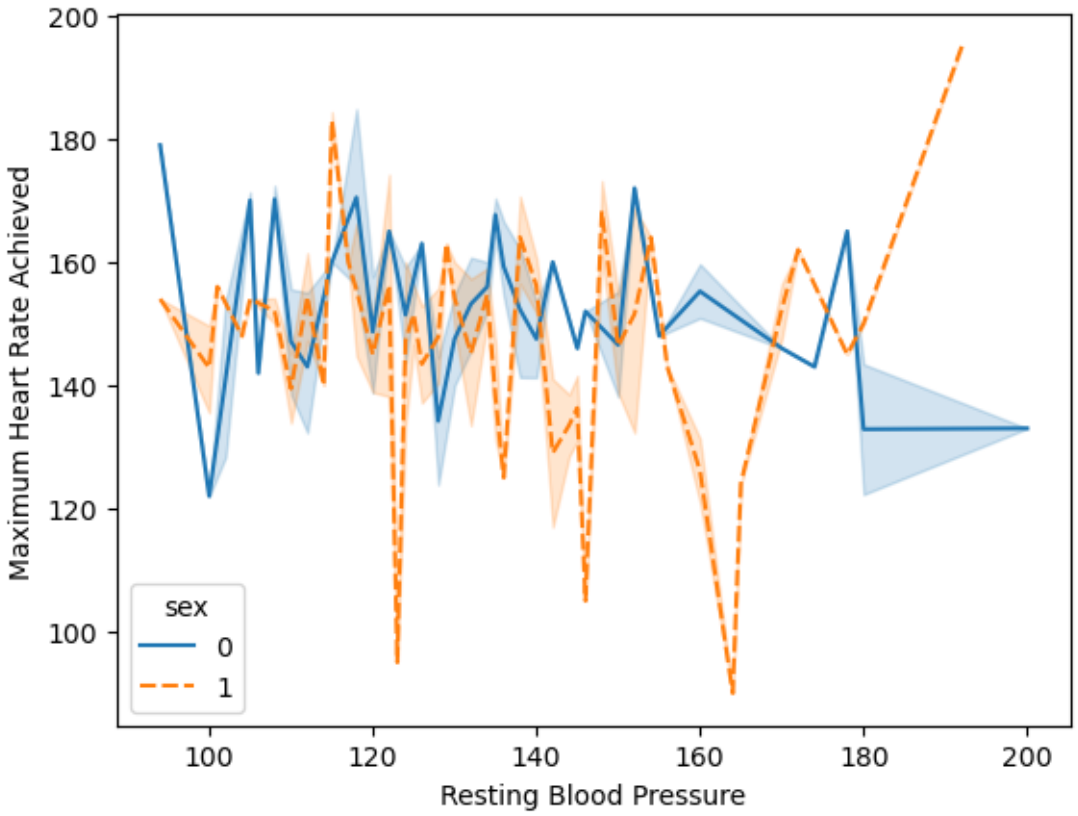
[27]:
```
import seaborn as so
heart_df.rename(columns = {"trestbps" : "Resting Blood Pressure", "thalach" :␣
↪"Maximum Heart Rate Achieved", "target" : "Heart Disease"}, inplace = True)
so.lineplot(data = heart_df["age"]);
```



[28]:
```
so.lineplot(data = heart_df, x = "Resting Blood Pressure", y = "Maximum Heart␣
↪Rate Achieved");
```



[29]:
```
so.lineplot(data = heart_df, x = "Resting Blood Pressure", y = "Maximum Heart␣
↪Rate Achieved", hue = "sex", style = "sex");
```

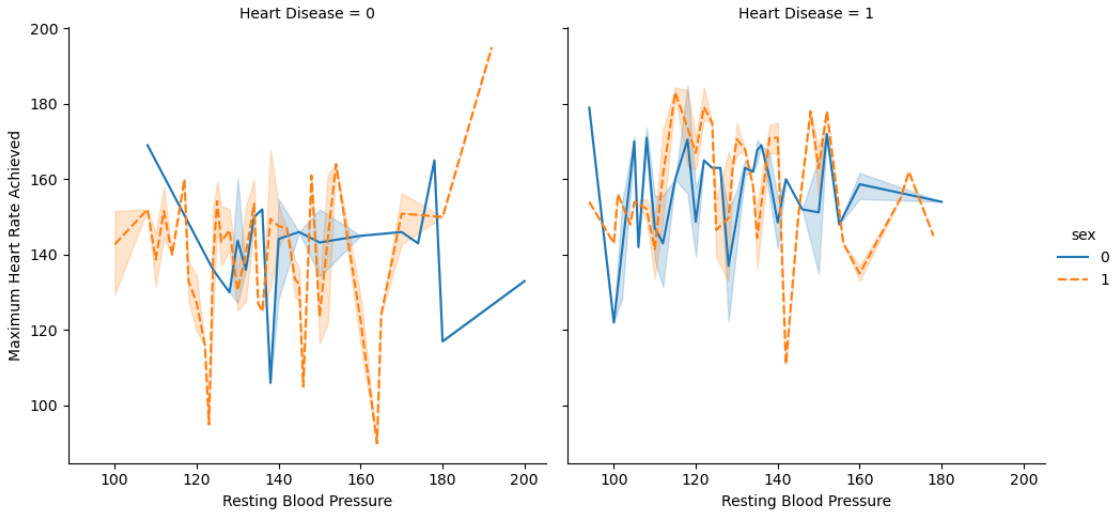


24 Relation Plot

Allows line plot grouping within an additional categorical variable.

```
so.relplot(data = heart_df, x = "Resting Blood Pressure", y = "Maximum Heart Rate Achieved", hu
```

[30]:
```
so.relplot(data = heart_df, x = "Resting Blood Pressure", y = "Maximum Heart␣
↪Rate Achieved", hue = "sex", style ="sex", kind = "line", col = "Heart␣
↪Disease");
```



25 Setting parameters

25.1 Set grid style

- darkgrid
- whitegrid
- dark
- white
- ticks

```
so.set_style("whitegrid")
```

25.2 Setting plot context

- paper
- notebook
- talk
- poster

```
so.set_context("poster")
```

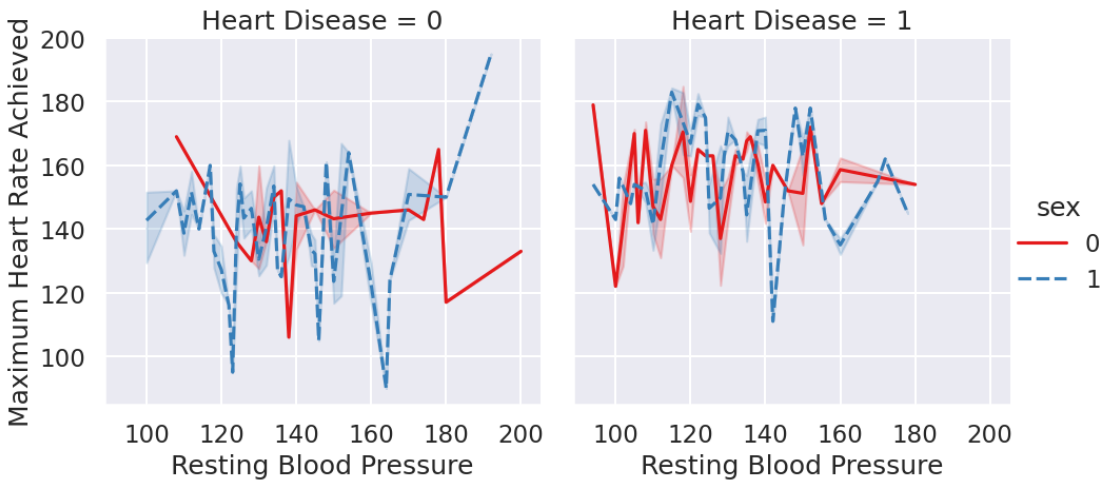
25.3 Set colors by selecting a desired palette (color map)

https://matplotlib.org/stable/gallery/color/colormap_reference.html ### Sample colormaps: Set1, Dark2, Accent, Pastel1, Reds, Greens, Blues

```
so.set_palette("Set1")
```

[31]:
```
so.set_style("darkgrid")
so.set_context("talk")
so.set_palette("Set1")
```

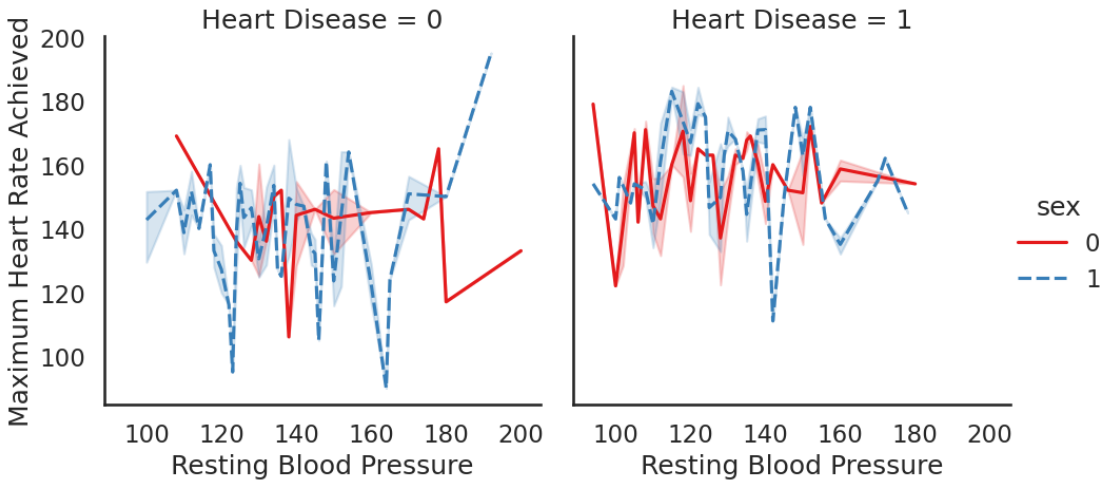
```
so.relplot(data = heart_df, x = "Resting Blood Pressure", y = "Maximum Heart␣
 ↪Rate Achieved", hue = "sex", style ="sex", kind = "line", col = "Heart␣
 ↪Disease");
```



26 Reset to the Seaborn defaults

```
so.set_theme()
```

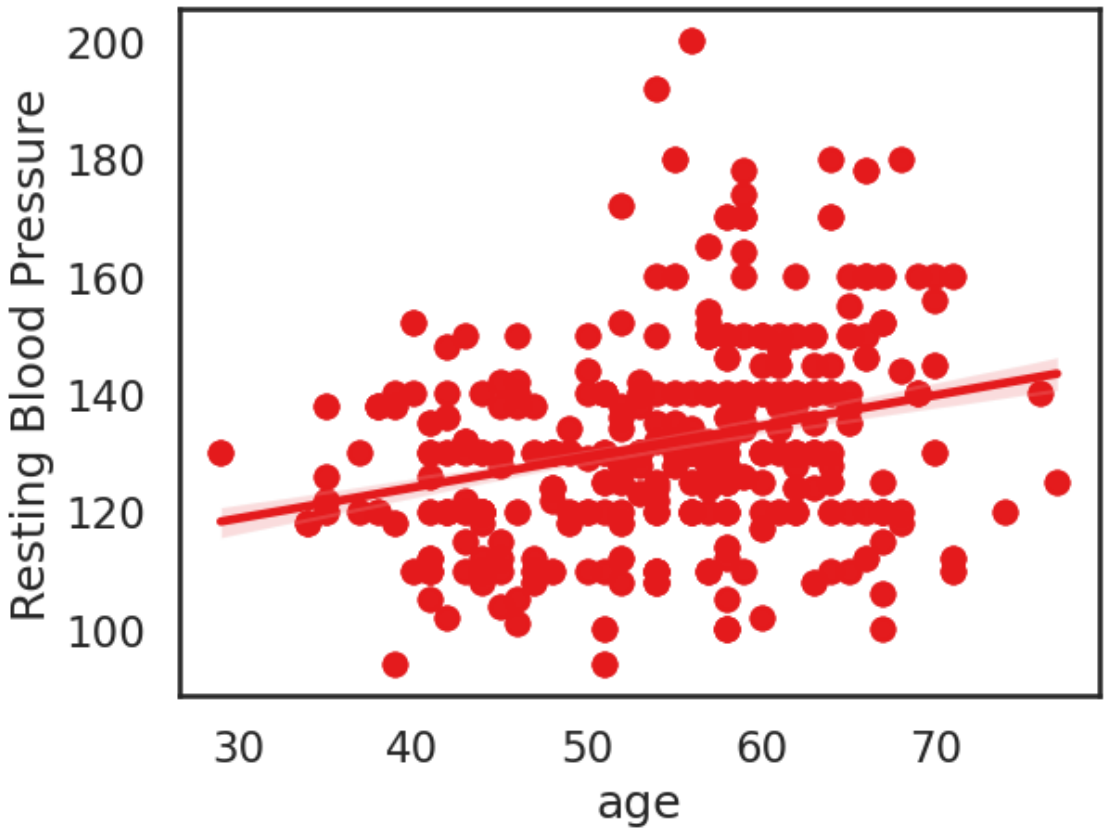
[32]:
```
so.set_theme(style = "white", context = "talk", palette = "Set1")
so.relplot(data = heart_df, x = "Resting Blood Pressure", y = "Maximum Heart␣
 ↪Rate Achieved", hue = "sex", style ="sex", kind = "line", col = "Heart␣
 ↪Disease");
```



27 Linear Regression Plot

```
so.regplot(data = heart_df, x = "age", y = "Resting Blood Pressure");
```

[33]:
```
so.regplot(data = heart_df, x = "age", y = "Resting Blood Pressure");
```



[34]:
```
heart_df
```

[34]:

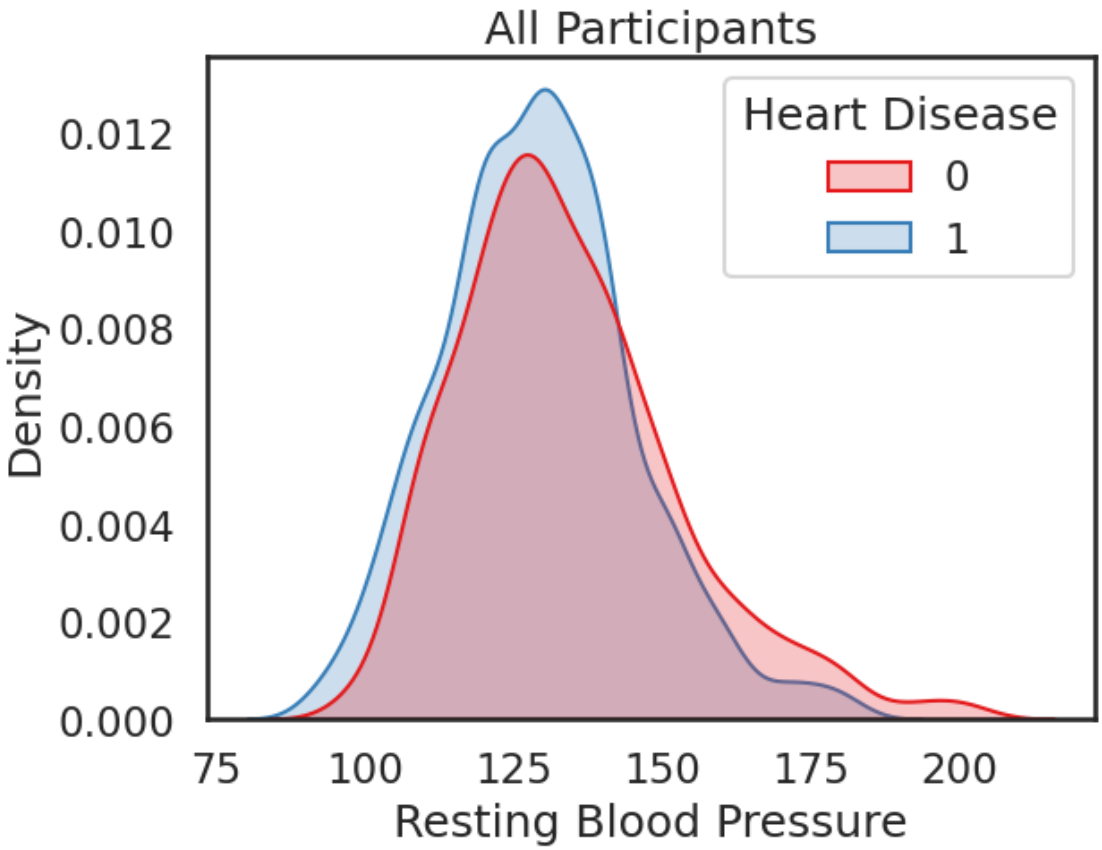
	age	sex	cp	Resting Blood Pressure	chol	fbs	restecg	\
0	52	1	0	125	212	0	1	
1	53	1	0	140	203	1	0	
2	70	1	0	145	174	0	1	
3	61	1	0	148	203	0	1	
4	62	0	0	138	294	1	1	
…	…	…	..	…	…	…	…	
1020	59	1	1	140	221	0	1	
1021	60	1	0	125	258	0	0	
1022	47	1	0	110	275	0	0	
1023	50	0	0	110	254	0	0	
1024	54	1	0	120	188	0	1	

```
      Maximum Heart Rate Achieved  exang  oldpeak  slope  ca  thal  \
0                             168      0      1.0      2   2     3
1                             155      1      3.1      0   0     3
2                             125      1      2.6      0   0     3
3                             161      0      0.0      2   1     3
4                             106      0      1.9      1   3     2
…                             …    …       …    …  ..   …
1020                          164      1      0.0      2   0     2
1021                          141      1      2.8      1   1     3
1022                          118      1      1.0      1   1     2
1023                          159      0      0.0      2   0     2
1024                          113      0      1.4      1   1     3

      Heart Disease
0                 0
1                 0
2                 0
3                 0
4                 0
…               …
1020              1
1021              0
1022              0
1023              1
1024              0

[1025 rows x 14 columns]
```

28 Density Plot

```
fig, ax = plt.subplots()
ax.set(title = "All Participants")
so.kdeplot(data = heart_df, x = "oldpeak", hue = "Heart Disease", fill = True);
```

[35]:
```
fig, ax = plt.subplots()
ax.set(title = "All Participants")
so.kdeplot(data = heart_df, x = "Resting Blood Pressure", hue = "Heart␣
 ↪Disease", fill = True);
```



[36]:
```
heart_df
```

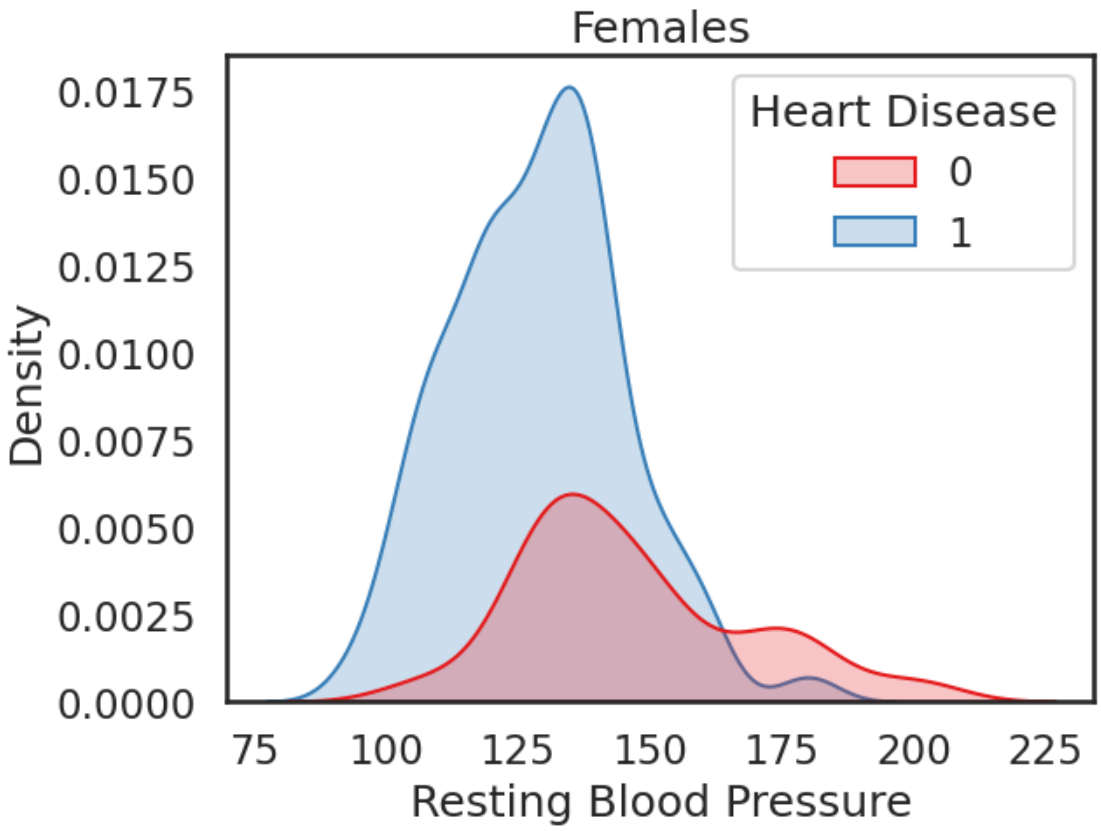
[36]:

	age	sex	cp	Resting Blood Pressure	chol	fbs	restecg	\
0	52	1	0	125	212	0	1	
1	53	1	0	140	203	1	0	
2	70	1	0	145	174	0	1	
3	61	1	0	148	203	0	1	
4	62	0	0	138	294	1	1	
…	…	…	..	…	…	…	…	
1020	59	1	1	140	221	0	1	
1021	60	1	0	125	258	0	0	
1022	47	1	0	110	275	0	0	
1023	50	0	0	110	254	0	0	
1024	54	1	0	120	188	0	1	

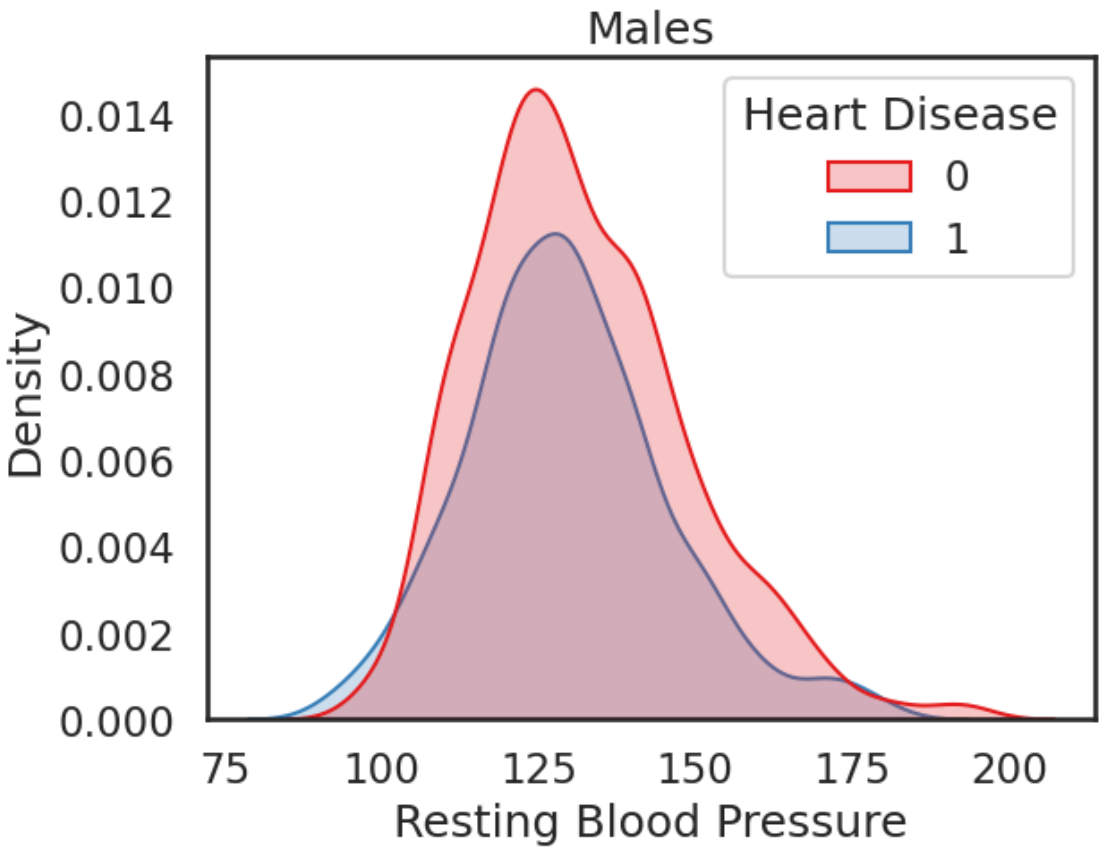
	Maximum Heart Rate Achieved	exang	oldpeak	slope	ca	thal	\
0	168	0	1.0	2	2	3	
1	155	1	3.1	0	0	3	
2	125	1	2.6	0	0	3	
3	161	0	0.0	2	1	3	

```
4                              106      0      1.9      1   3     2
…                              …    …       …    …  ..   …
1020                           164      1      0.0      2   0     2
1021                           141      1      2.8      1   1     3
1022                           118      1      1.0      1   1     2
1023                           159      0      0.0      2   0     2
1024                           113      0      1.4      1   1     3

      Heart Disease
0                 0
1                 0
2                 0
3                 0
4                 0
…               …
1020              1
1021              0
1022              0
1023              1
1024              0

[1025 rows x 14 columns]
```

[37]:
```
fig, ax = plt.subplots()
ax.set(title = "Females")
so.kdeplot(data = heart_df, x = heart_df.loc[heart_df["sex"] == 0, "Resting␣
 ↪Blood Pressure"], hue = "Heart Disease", fill = True);
```

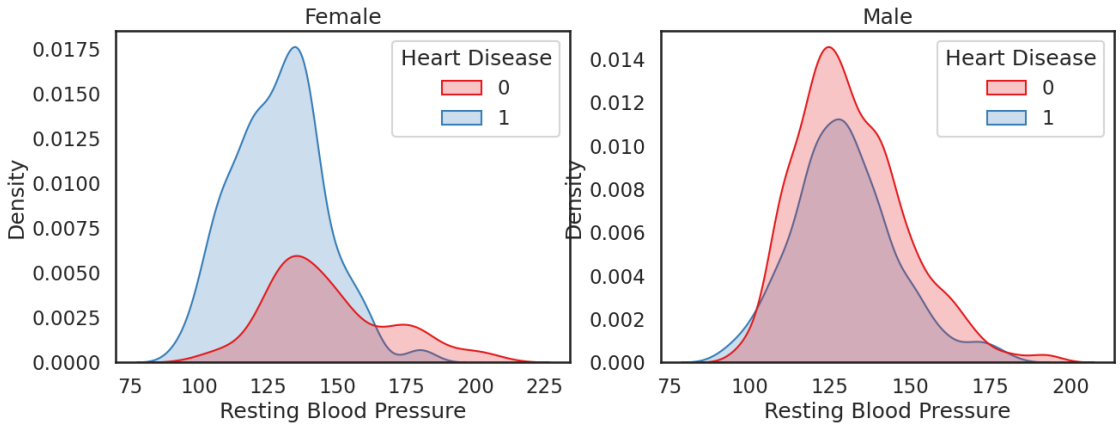


[38]:
```
fig, ax = plt.subplots()
ax.set(title = "Males")
so.kdeplot(data = heart_df, x = heart_df.loc[heart_df["sex"] == 1, "Resting␣
 ↪Blood Pressure"], hue = "Heart Disease", fill = True);
```



29 Combine the plots and scale

[39]:
```
fig, ax = plt.subplots(1, 2, figsize = (15, 5))
ax[0].set(title = "Female")
ax[1].set(title = "Male")

so.kdeplot(data = heart_df, x = heart_df.loc[heart_df["sex"] == 0, "Resting␣
 ↪Blood Pressure"],
           hue = "Heart Disease", fill = True, ax = ax[0]);
so.kdeplot(data = heart_df, x = heart_df.loc[heart_df["sex"] == 1, "Resting␣
 ↪Blood Pressure"],
           hue = "Heart Disease", fill = True, ax = ax[1]);
```

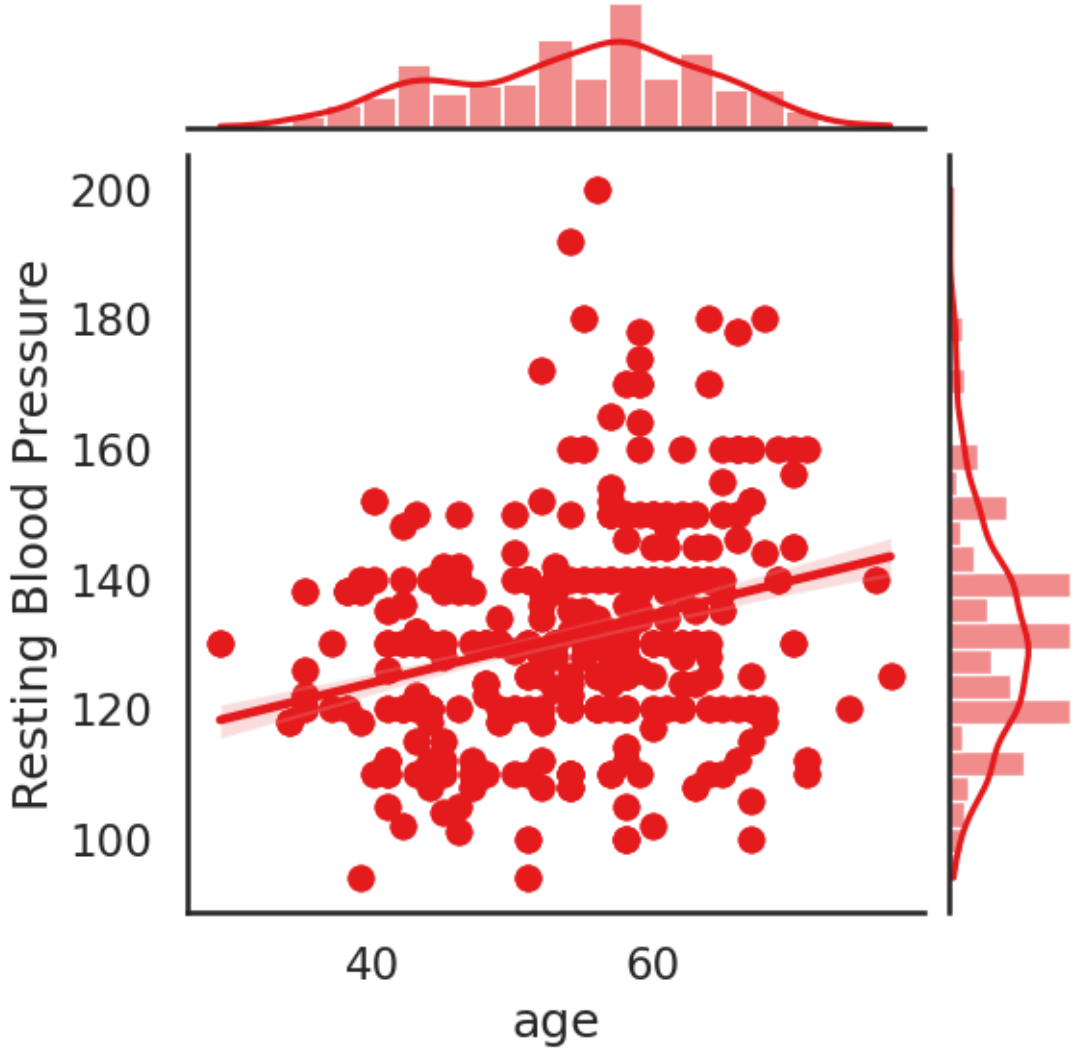


30 Joint Plots

Histogram and Regression

```
so.jointplot(data = heart_df, x = "age", y = "Resting Blood Pressure", kind = "reg");
```

[40]:
```
so.jointplot(data = heart_df, x = "age", y = "Resting Blood Pressure", kind =␣
↪"reg");
```



[ ]:
```
so.pairplot(data = heart_df, hue = "Heart Disease");
```

[ ]:
```

```